

Example Missions

Five canonical MOOS-IvP example missions covering loitering, depth-plane operation, dynamic behavior spawning, standby helm operation, and collision avoidance.

Included MIT MOOS-IvP PDF sources:

- chap_xmiss_charlie.pdf
- chap_xmiss_delta.pdf
- chap_xmiss_echo.pdf
- chap_xmiss_kilo.pdf
- chap_xmiss_berta.pdf

The Charlie Mission

Fall 2024

Michael Benjamin, mikerb@mit.edu
Department of Mechanical Engineering
MIT, Cambridge MA 02139
project-pavlab/chapters/chap_xmiss_charlie

1	The Charlie Mission	1
1.1	Launching the Charlie Mission	1
1.2	Topic #1: Hierarchical Mode Declarations in the Charlie Mission	2
1.3	Topic #2: The Loiter Behavior	3
1.4	Topic #3: The StationKeep Behavior	5
1.5	Suggestions for Tinkering in the Charlie Mission	6

Contents

1 The Charlie Mission

- Behaviors:** Loiter, StationKeep, Waypoint
- MOOS Apps:** `pHelmIvP`, `pLogger`, `uSimMarine`, `pMarinePID`, `pNodeReporter`, `pMarineViewer`, `uTimerScript`
- Primary Topics:** (1) Hierarchical Mode Declarations
(2) The Loiter behavior
(2a) Loiter traversal - Clockwise vs. counter-clockwise
(2b) Loiter polygon shapes, hexagons, ellipses
(2c) Loiter dynamic changes to the polygon center position
(2d) Loiter robustness to periodic external forces
(3) The StationKeep Behavior
- Side Topics:** (1) `uTimerScript` is used to simulate wind gusts, external forces
(2) Use of `pMarineViewer` action buttons and action pull-down menu

1.1 Launching the Charlie Mission

The charlie mission may be launched from the command line in the following manner:

```
$ cd moos-ivp/missions/s3_charlie/  
$ ./launch.sh --warp=10
```

This should bring up a `pMarineViewer` window like that shown in Figure 2, with a single vehicle, *henry*, initially in the PARK mode. After hitting the DEPLOY button in the lower right corner, the vehicle enters the "LOITERING" mode and begins to proceed to the polygon shown. To get a quick feel for what's possible in this simulation, the following are some things to try. (a) Click on the "CWISE" and "CCWISE" buttons to change the direction of loitering around the polygon. (b) Click

the RETURN and DEPLOY buttons repeatedly to see the vehicle switch between two primary modes. (c) Select the "WIND_GUSTS=true" option in the ACTION pull-down menu to see the vehicle adjust to random periodic external forces. (d) Select the "STATION_KEEP=true" option in the ACTION pull-down menu to see how the vehicle station keeps with simulated wind gust. (e) Let the vehicle return to its return point and watch how it automatically switches to the station keeping mode, and then re-deploy it. (f) Select the ellipse polygon from the ACTION pull-down menu to see a different loiter shape. (g) Select the different center points for the loiter ellipse from the ACTION pull-down menu to see just the location of the loiter polygon change and the vehicle adjust.

1.2 Topic #1: Hierarchical Mode Declarations in the Charlie Mission

The Charlie mission is organized by hierarchical mode declarations into four modes: INACTIVE, LOITERING, RETURNING, and STATION-KEEPING. The mode declarations are given at the top of the charlie.bhv file and shown again in Figure 1. The pMarineViewer application by default shows the vehicle's mode alongside the vehicle name, and when the simulation is first launched, the Charlie vehicle is shown on the screen with the mode PARK.

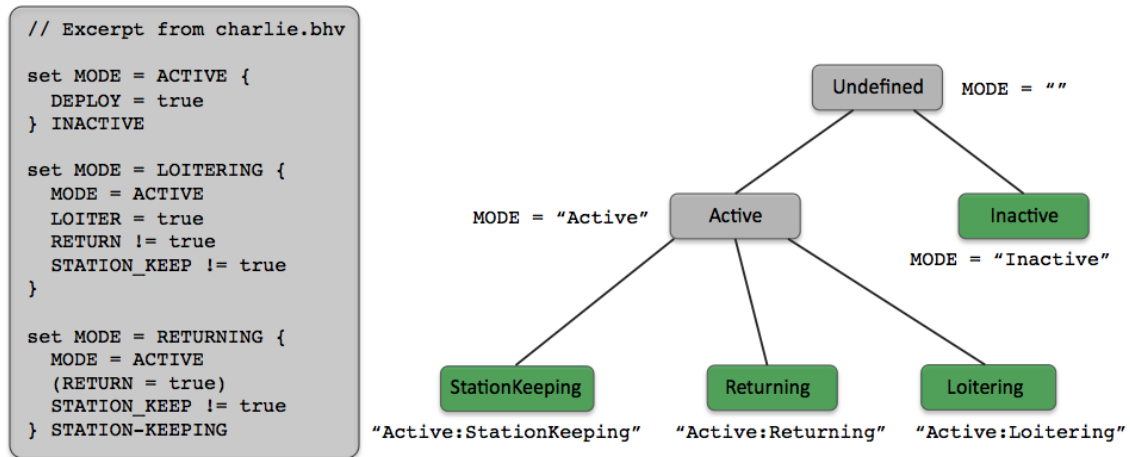


Figure 1: **Charlie Mission Mode Declarations:** The helm, when in drive, will be in one of the four modes indicated by the leaf nodes on the tree. The hierarchical mode structure means, for example, that when the vehicle is in the "RETURNING" mode, it is also considered to be in the "ACTIVE" mode. The vehicle mode, by default, is displayed alongside the vehicle name in the pMarineViewer window.

The "PARK" mode displayed with the vehicle when the simulation is launched is actually the high level helm state. As described in Section ??, the helm has a higher level *helm state* which is either PARK or DRIVE, analogous to a car in either the "Park" or "Drive" modes. When the Charlie mission is first launched, the helm is not yet in drive, and is put into drive when the DEPLOY button is hit. This button is configured in the pMarineViewer configuration block to make several MOOS pokes with a single button click. It posts MOOS_MANUAL_OVERRIDE=false, to put the vehicle in the DRIVE helm state. It posts DEPLOY=true to put the helm in the ACTIVE mode, and it posts LOITER=true to put the helm in the ACTIVE:LOITERING mode.

The Charlie mission is configured to allow for easy transition between the other modes via the pMarineViewer interface. The vehicle may be put into the STATION-KEEPING mode at any time via the

`STATION_KEEP=true` option in the ACTION pull-down menu in `pMarineViewer`. This does nothing more than make the `STATION_KEEP=true` post to the MOOSDB. The same could have been accomplished by `uPokeDB` for example:

```
$ cd moos-ivp/missions/s3_charlie/
$ uPokeDB charlie.moos STATION_KEEP=true
```

Referring to the mode declarations in Figure 1, the `LOITERING` and `RETURNING` modes both have the condition `STATION_KEEP != true`, so both of those mode requirements fail to be satisfied. The `STATION-KEEPING` mode is the default mode, i.e., the "else" mode when the requirements of the `RETURNING` mode are not met. The `STATION_KEEP=true` posting could also have been made by another MOOS process, for example, connected to an acoustic modem, an Iridium satellite, or wifi interface.

The helm mode is communicated to `pMarineViewer` from the helm via the MOOS variable `IVPHELM_SUMMARY`. This variable is parsed and the mode is rendered next to the vehicle name in the viewer. This rendering may be turned off by toggling through the "name options", with the 'n' key, or setting the `vehicles_name_viewable` parameter in the `pMarineViewer` configuration block to one of the values described in the "Vehicles Pull-Down Menu" Section of [?].

1.3 Topic #2: The Loiter Behavior

The `LOITERING` mode in the Charlie mission is characterized by the `BHV.Loiter` behavior, and we explore some of capabilities and options for this behavior here. The behavior configuration, in file `charlie.bhv` is shown in Listing 1 below. The first configuration block, in lines 3-6, are for parameters defined generally for all IvP Helm behaviors, and the second block, in lines 8-16, are for parameters defined explicitly for the Loiter behavior.

Listing 1.1: Configuration of the Loiter behavior in the Charlie mission, from the file `charlie.bhv`.

```
1  //-----
2  Behavior = BHV_Loiter
3  {
4      name      = loiter
5      priority  = 100
6      condition = MODE==LOITERING
7      updates   = UP_LOITER
8
9          speed = 1.3
10         clockwise = false
11     capture_radius = 4.0
12     nm_radius = 15.0
13     polygon = format=radial, x=0, y=-75, radius=40, pts=6, snap=1,
14     visual_hints = nextpt_color=white, nextpt_lcolor=khaki
15     visual_hints = edge_color=blue, vertex_color=yellow
16     visual_hints = edge_size=1, vertex_size=2, label=LOITER_POLYGON
17 }
```

When the vehicle is first deployed, it heads to the hexagon shaped polygon shown in Figure 2, configured in line 13. It traverses the polygon in a counter-clockwise manner due to the configuration in line 10. The Loiter behavior maintains an internal mode, the `"acquire_mode"` which is true when

the vehicle is not sufficiently close to the polygon, defined by the `acquire_dist` parameter which is by default 10 meters. It also keeps track of how many vertex arrivals come by way of achieving the capture radius, and how many by way of the non-monotonic radius. These internal state variables are summarized by the Loiter behavior by publishing a variable `LOITER_REPORT`. An example might look like:

```
LOITER_REPORT = "index=2,capture_hits=12,nonmono_hits=3,acquire_mode=false"
```

In the Charlie mission, `pMarineViewer` is configured to scope on this variable by default, and the evolving `LOITER_REPORT` may be monitored as the vehicle progresses around the polygon.

1.3.1 Loiter Traversal - Clockwise vs. Counter-Clockwise

The traversal direction of the Loiter behavior is by default clockwise and can be set to counter-clockwise with the parameter `clockwise=false`. In the Charlie mission, the traversal direction can be changed by selecting `UP_LOITER = clockwise=true` and `UP_LOITER=clockwise=false` from the ACTION pull-down menu. This is a good way to observe the algorithm used by the behavior to acquire the polygon when sufficiently far inside the polygon. The traversal direction could also have been affected by any other MOOS application capable of executing the above two MOOS pokes, including uPokeDB, or any application connected to a communications device.

1.3.2 Loiter Polygon Shapes, Hexagons, and Ellipses

The only restriction on the shape traversed by the Loiter behavior is that it be a convex polygon. The typical specification of the polygon is shown in line 13, via the "radial" syntax, which accepts a center position ("`x=0, y=-75`"), a distance from the center point to each vertex ("`radius=40`"), the number of points in the polygon, ("`pts=6`"), and a snap value, which rounds the vertex positions to, in this case, the nearest whole meter ("`snap=1`"). Ellipse shapes are supported with a string such as:

```
polygon = format=ellipse, x=110, y=-75, degs=150, pts=14, major=80, minor=20
```

The above ellipse specification is selectable in the Charlie mission via the ACTION pull-down menu. If selected while in the loiter mode, the vehicle immediately begins traversing to this new polygon, as shown in Figure 3.

1.3.3 Loiter Dynamic Changes to the Polygon Position

In addition to the an outright change to the loiter polygon as described above, which includes shape changes, number of vertices and position, there are a few methods for just changing the polygon position while leaving the other characteristics in tact. The Loiter behavior accepts a parameter for just the center position. The following two such assignments are selectable in the ACTION pull-down menu in the `pMarineViewer` for the Charlie mission:

```
center_assign = 40,-40  
center_assign = x=100, y=-80
```

Try selecting either of these options, with the polygon set to either the hexagon or ellipse and confirm that only the position changes. One other method for affecting the polygon position is via the `center_activate` parameter, which is `false` by default. When set to `true`, the polygon center is set to be the vehicle's present position when the Loiter behavior enters the running state. In the Charlie mission, try selecting `center_activate=true` from the ACTION pull-down menu. If already in the LOITERING mode, then nothing happens immediately. If and when the helm exits and returns to the LOITERING mode, for example after clicking the RETURN button, and then the DEPLOY button again, note that the loiter polygon is centered on the vehicle's present position. The `center_activate` feature is particularly useful when the Loiter behavior is used more or less as a station keeping behavior and one wants to be able to command the vehicle to loiter at the present position at any given time.

1.3.4 Loiter Robustness to Periodic External Forces

When the Loiter behavior is active and the vehicle is proceeding around the polygon vertices, it is proceeding more or less like the Waypoint behavior (both behaviors share the same WaypointEngine class as a sub-component). Things get interesting when the vehicle is approaching the polygon from the outside or from a distance internally (for example when the `center_activate` parameter set to `true`). To test and demonstrate this robustness, the Charlie mission is configured with the `uTimerScript` utility to generate periodic random forces to simulate wind gusts, to push the vehicle off the loiter polygon in unpredictable ways. An example is shown in Figure 4.

The simulated wind gusts may be activated by selecting the `WIND_GUSTS=true` option from the `pMarineViewer` ACTION pull-down menu. The details of the simulated forces can be found in the `uTimerScript` configuration block in the `charlie.moos` file. The script works by posting external force vectors to the MOOS variable `USM_DRIFT_VECTOR_ADD`, which is read by the `uSimMarine` application to alter the prevailing external force vector, which by default has a magnitude of zero. The syntax of the `uTimerScript` script to implement the drift effects in this example are described in more detail in the "A Script as a Proxy for Simulating Random Wind Gusts" Section of the `uTimerScript` documentation, [?].

1.4 Topic #3: The StationKeep Behavior

The STATION-KEEPING mode in the Charlie mission is characterized by the `BHV_StationKeep` behavior, and we explore some of capabilities and options for this behavior here. The behavior configuration, in file `charlie.bhv` is shown in Listing 2 below. The first configuration block, in lines 3-5, are for parameters defined generally for all IvP Helm behaviors, and the second block, in lines 7-11, are for parameters defined explicitly for the Loiter behavior.

Listing 1.2: Configuration of the StationKeep behavior in the Charlie mission, from the file charlie.bhv.

```

0  //-----
1  Behavior = BHV_StationKeep
2  {
3      name      = station-keep
4      priority  = 100
5      condition = MODE==STATION-KEEPING
6  }
```

```

7   center_activate = true
8   inner_radius = 5
9   outer_radius = 10
10  outer_speed = 1.0
11  swing_time = 0
12 }

```

1.4.1 Putting the Vehicle into the Station Keeping Mode

The vehicle is put into the station keeping mode by posting `STATION_KEEP=true` to the MOOSDB. This results in the helm mode, represented by the MOOS variable `MODE`, being set to station keeping. The hierarchical mode declarations used in the Charlie mission are declared in the top of the `charlie.bhv` file and were shown in Figure 1. The StationKeep behavior is conditioned on `MODE==STATION-KEEPPING`, on line 5 above. The StationKeep behavior may be configured to station keep at a specified point in water (with the `station_pt` parameter), or it may be configured to station keep wherever it happens to be when it enters or re-enters the running state, as it is configured in the Charlie mission by virtue of line 7 above. In the Charlie mission, `pMarineViewer` is configured with four on-screen buttons. For more info, see the "Command-and-Control" section of the `pMarineVeiw` documentation, [?]. Two of the buttons are used for toggling the `STATION_KEEP` variable to the MOOSDB.

1.4.2 What is Happening in the Station Keeping Mode

The station keeping behavior (the only running behavior in this mode) attempts to keep the vehicle within a certain distance, given by the `inner_radius` parameter, to a center point. Inside this radius, it simply lets the vehicle drift. Outside the radius it sets a heading and speed to drive the vehicle back to the center point. The speed decreases as it approaches the inner circle, and is at its highest speed (set by the `outer_speed` parameter on line 10 above) when its range to the center point is greater than that given by the `outer_radius` distance. It also makes two postings to the `VIEW_POLYGON` object to represent the circles implied by the `inner_radius` and `outer_radius` parameters, as shown in Figure 5.

1.5 Suggestions for Tinkering in the Charlie Mission

- Station keeping is trivial if the vehicle doesn't drift. Try turning on the artificial wind gusts available in the Charlie mission, courtesy of the `uTimerScript` script, available by selecting `WIND_GUSTS=true` from the Action pull-down menu. The vehicle performance is a bit more interesting now in the station keeping mode.
- Try configuring the simulator, `uSimMarine`, to reflect a vehicle that takes much more time to come to a full stop in the water. This can be done by setting `deceleration=0.1` in the `uSimMarine` configuration block, or by posting `USM_DECELERATION=0.1` to the MOOSDB once the mission is running. Note that when the vehicle is put into station keeping mode, its station point is set to the point where it is when it enters this mode. Since the vehicle takes so long to slow down, it immediately drifts out of the inner radius and turns around 180 degrees. This is a typical situation seen in the field. This can be countered a bit by setting the `swing_time` parameter in the StationKeep behavior. This parameter is the number of elapsed seconds

after the vehicle enters the station keeping mode before the station keeping behavior marks its present position as the point to station keep around. Try setting `swing_time=5` and re-running the above to see the difference.

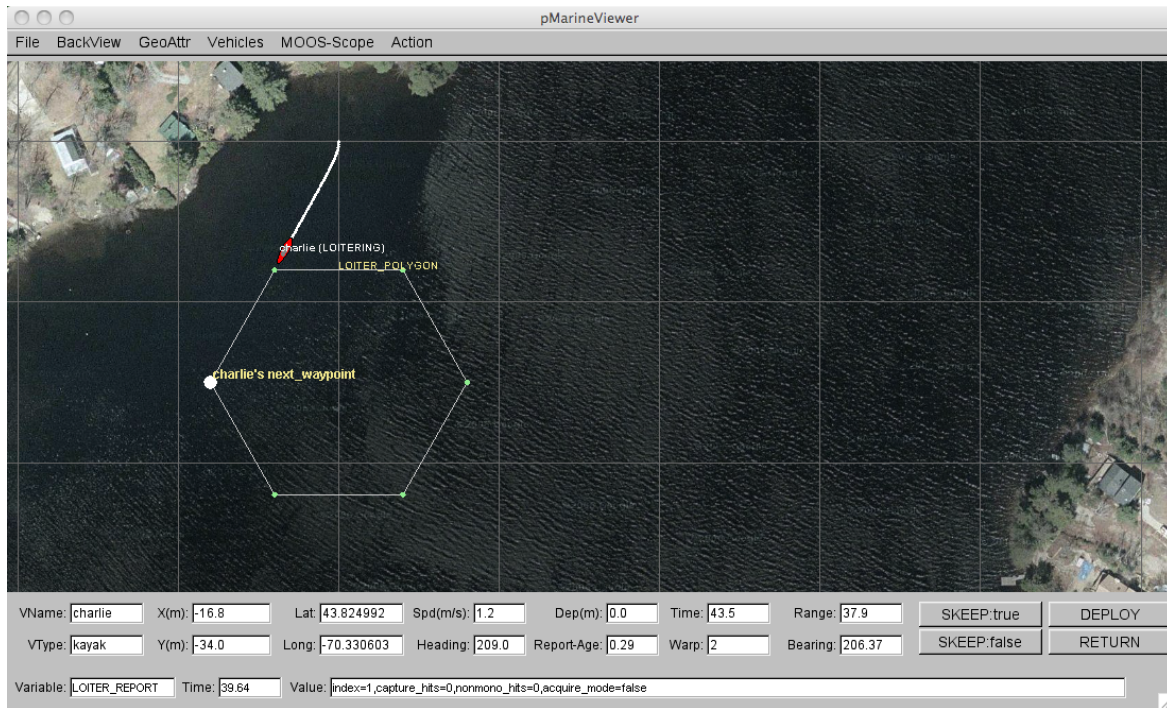


Figure 2: **The Charlie Mission (1):** The vehicle "charlie" proceeds to a loiter polygon, traversing in a counter-clockwise manner to the first waypoint labelled "charlie's next waypoint".

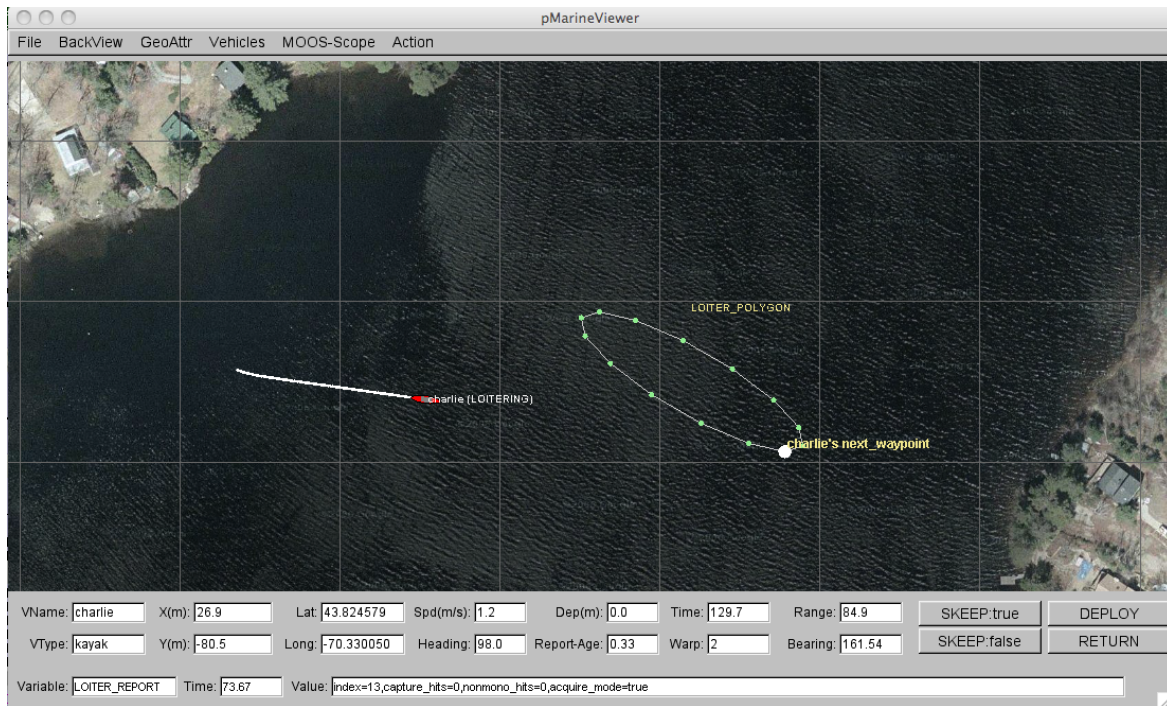


Figure 3: **The Charlie Mission (2):** The loiter traversal polygon for the vehicle "charlie" has been dynamically changed to an ellipse via a selection from the ACTION pull-down menu.

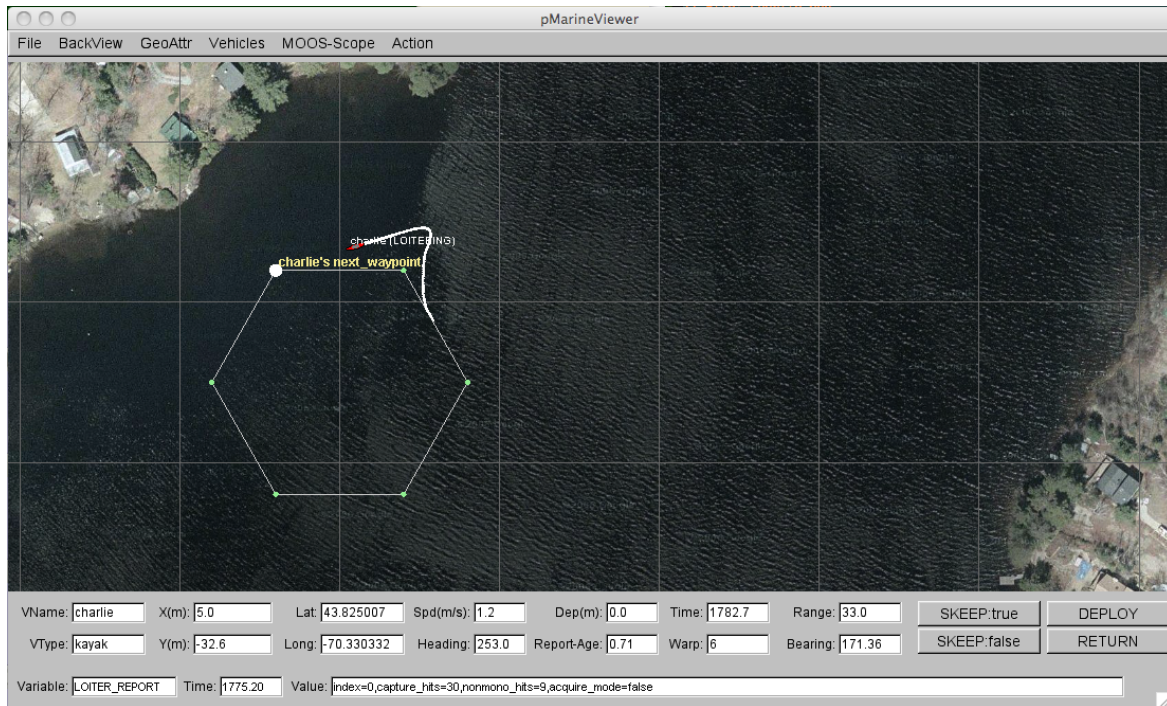


Figure 4: **The Charlie Mission (3):** The vehicle, "charlie" recovers from a wind gust and proceeds to re-enter the polygon trajectory at the tangential vertex labelled "charlie's next waypoint".

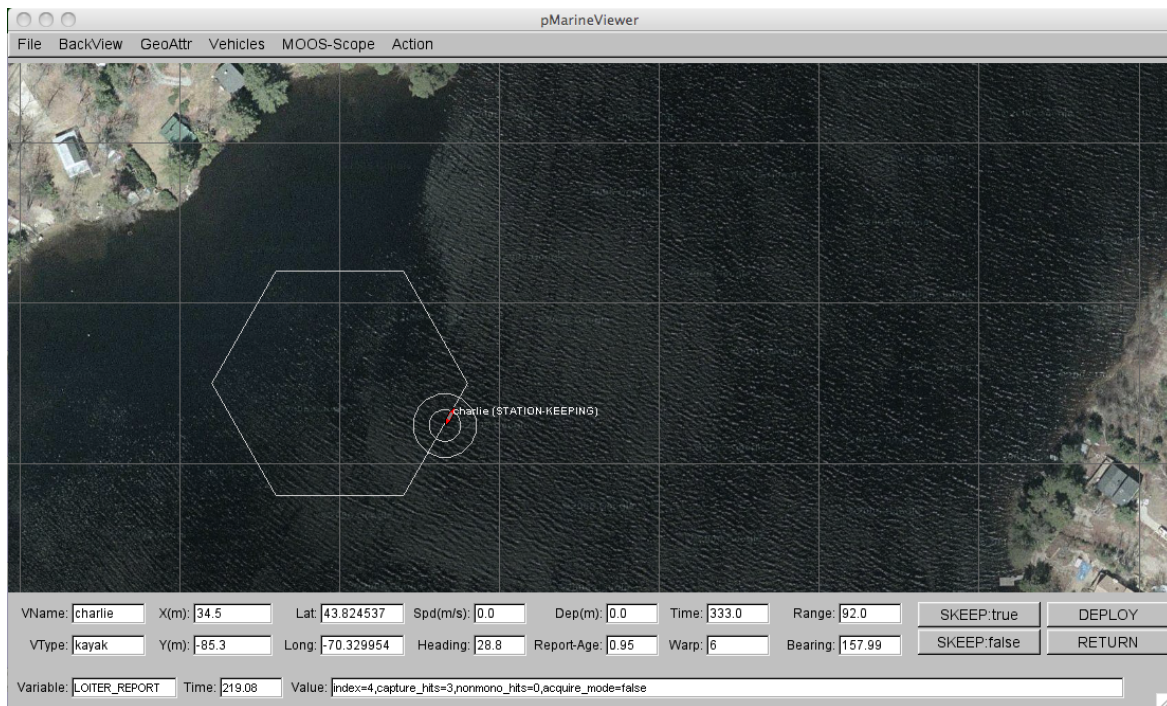


Figure 5: **The Charlie Mission (4):** The vehicle "charlie" is put into the station keeping mode. The two rings around the vehicle are the **inner radius** and **outer radius** parameters.

The Delta Mission

Fall 2024

Michael Benjamin, mikerb@mit.edu
Department of Mechanical Engineering
MIT, Cambridge MA 02139
project-pavlab/chapters/chap_xmiss_delta

1	The Delta Mission	1
1.1	Overview of the Delta Mission Components and Topics	1
1.2	Launching the Delta Mission	1
1.3	Topic #1: Configuring the Helm for Operation at Depth	2
1.4	Topic #2: The ConstantDepth Behavior	3
1.5	Topic #3: The PeriodicSurface Behavior	3
1.6	Topic #4: The Waypoint Behavior with Survey Patterns	4
1.7	Topic #4: Using pMarineViewer with Geo-referenced Mouse Clicks	5
1.8	Suggestions for Further Experimenting with the Delta Mission	6

Contents

1 The Delta Mission

The Delta example mission is used to illustrate the operation of a a vehicle in the depth plane (an underwater as opposed to surface vehicle), the illustration of the PeriodicSurface behavior, the use of the Waypoint behavior with survey patterns, and the use of the `pMarineViewer` application to send field-control commands to alter the mission as it unfolds. The vehicle will initially deploy using the Loiter behavior, and will periodically come to the surface. If commanded by the user, the vehicle will break off from the Loiter behavior to execute a survey pattern, after which it will resume loitering at its original location.

1.1 Overview of the Delta Mission Components and Topics

Behaviors:	Waypoint, Loiter, PeriodicSurface, ConstantDepth
MOOS Apps:	<code>pHelmIvP</code> , <code>pLogger</code> , <code>uSimMarine</code> , <code>pMarinePID</code> , <code>pNodeReporter</code> , <code>pMarineViewer</code> <code>uTimerScript</code>
Primary Topics:	(1) Configuring the Helm for operation at depth (2) The ConstantDepth behavior (3) The PeriodicSurface behavior (4) The Waypoint behavior with survey patterns (5) Using pMarineViewer with Geo-referenced mouse clicks
Side Topics:	(1) <code>uTimerScript</code> is used to simulate a UUV receiving a GPS update.

1.2 Launching the Delta Mission

The delta mission may be launched from the command line:

```
$ cd moos-ivp/missions/s4_delta/  
$ ./launch.sh --warp=10
```

This should bring up a `pMarineViewer` window like that shown in Figure 1, with a single vehicle, "dudley", initially in the PARK helm state. See Section ?? for more on the helm state. After hitting the DEPLOY button in the lower right corner, the vehicle enters the DRIVE state, and the LOITERING helm mode, and begins to proceed along the waypoints as shown. See Section ?? for more on more on the helm mode, vs. the helm state. The mode declarations for the Delta mission are defined at the top of the `delta.moos` file, and amount to the following simple hierarchy:

```
o ROOT  
|--o INACTIVE  
|--o ACTIVE  
    |--o RETURNING  
    |--o SURVEYING  
    |--o LOITERING
```

In the initial helm mode after deployment, the LOITERING mode, the helm is running the Loitering behavior, Section ??, the ConstantDepth behavior, Section ??, and the PeriodicSurface behavior, Section ?. It will stay in this mode indefinitely until it is either commanded to return or break off to another region to conduct a survey pattern. In the LOITERING mode the vehicle will periodically come to the surface for a GPS fix, presumably to correct for accumulated navigation error. The `uTimerScript` utility is used to simulate the event of receiving a GPS fix. This script is described in more detail in the `uTimerScript` documentation in the Section "A Script Used as Proxy for an On-Board GPS Unit".

In the SURVEYING mode, the Waypoint behavior becomes active and will execute a survey lawn-mower type pattern, as shown in Figures 2 and 4. In this mode, the periodic surfacing for GPS fixes is suppressed until the pattern is completed. The SURVEYING mode is entered whenever the MOOS variable SURVEY is set to true. In this mission, `pMarineViewer` is configured to toggle the SURVEY MOOS variable with on-screen buttons, and it is configured to accept mouse clicks which not only put the vehicle into the SURVEYING mode, but also accept the location of the mouse click as the center location of a predefined survey pattern. This kind of `pMarineViewer` configuration is discussed in Section 1.6.

1.3 Topic #1: Configuring the Helm for Operation at Depth

The Delta mission is configured to simulate a UUV and the helm is configured to produce decisions about the decision variables *heading*, *speed*, as well as *depth*. Adding *depth* to the helm decision space is done in the helm configuration block as shown below in Listing 1, taken from the `delta.moos` mission file. In this case, the *depth* decision space is defined on line 13 with a range from 0 to 500 meters, with a resolution of 1 meter. See Section ?? for more on configuring the helm decision space with the `domain` parameter.

Listing 1.1: Configuring pHelmIvP in the Delta mission to use behaviors concerning vehicle depth.

```
1 // pHelmIvP config block  
2  
3 ProcessConfig = pHelmIvP
```



```

4 {
5   AppTick      = 4
6   CommsTick    = 4
7
8   behaviors    = delta.bhv
9   verbose      = quiet
10  domain       = course:0:359:360
11  domain       = speed:0:4:21
12  domain       = depth:0:500:501
13 }

```

In this example, the *depth* decision variable is a mandatory variable, along with *heading* and *speed*. This means that, on any given helm iteration, there *must* be a decision made about depth, or else the helm will post a helm error. How does the helm ensure that a depth decision is always part of any decision? The helm must be configured such that a behavior that reasons about *depth* is always active regardless of the helm mode. In the Delta mission, the three primary modes, LOITERING, SURVEYING, RETURNING, are all sub-modes of the ACTIVE mode, as shown above. The ConstantDepth behavior is configured to be running whenever the helm is in the Active mode.

1.4 Topic #2: The ConstantDepth Behavior

The ConstantDepth behavior is used in the Delta mission to keep the vehicle a prescribed depth while it is transiting, surveying, and loitering. For simplicity a single ConstantDepth behavior is used, but a different behavior instance could also be used for each vehicle mode. The behavior configuration, in file `delta.bhv` is shown in Listing 2 below. The first part of the configuration block, in lines 3-6, are for parameters defined generally for all IvP Helm behaviors, and the second part, on line 8, is a parameter defined for the ConstantDepth behavior.

Listing 1.2: The ConstantDepth behavior in the Delta mission, from the file `delta.bhv`.

```

1 Behavior = BHV_ConstantDepth
2 {
3   name      = bhv_const_depth
4   pwt       = 100
5   duration  = no-time-limit
6   condition = MODE==ACTIVE
7
8   depth = 15
9 }

```

The ConstantDepth behavior’s primary parameter is the `depth` parameter on line 8. This behavior does make provisions for providing a range of depths, or a more gradually degrading utility function when this behavior is used to work with other behaviors reasoning about depth. In this mission however, the depth is mostly non-contentious between behaviors. The exception is when the PeriodicSurface behavior is active, in which case the priority weight for the PeriodicSurface behavior is simply set to suppress the ConstantDepth behavior.

1.5 Topic #3: The PeriodicSurface Behavior

The PeriodicSurface behavior, described generally in Section ??, is used in the Delta mission to bring the vehicle to the surface for GPS fixes periodically, to simulate the need for occasional

navigation corrections. The behavior configuration, in file `delta.bhv` is shown in Listing 3 below. The first part of the configuration block, in lines 3-6, is for parameters defined generally for all IvP Helm behaviors, and the second part, in lines 8-13, is for parameters defined for the PeriodicSurface behavior.

Listing 1.3: The PeriodicSurface behavior in the Delta mission, from the file delta.bhv.

```

1 Behavior = BHV_PeriodicSurface
2 {
3     name          = bhv_periodic_surface
4     pwt           = 1000
5     condition     = (MODE == LOITERING) or (MODE == RETURNING)
6     condition     = PSURFACE = true
7
8         period = 120
9     zero_speed_depth = 2
10    max_time_at_surface = 60
11        ascent_speed = 1.0
12        ascent_grade = fullspeed
13        mark_variable = GPS_UPDATE_RECEIVED
14 }
```

The PeriodicSurface behavior outputs an objective function solely on the *depth* decision variable. Note the priority weight in line 4 in Listing 3 is set to 1000, in comparison to the priority weight set for the ConstantDepth behavior on line 4 in Listing 2. When the PeriodicSurface behavior is running, it may or may not be active and producing an objective function over the *depth* decision variable. When it *is* active, the influence on *depth* simply overrules the influence from the ConstantDepth behavior due to these relative priority weights. For this reason, the ConstantDepth behavior may be conditioned on all active modes as in line 6 of Listing 2.

Note the parameter setting on line 13 above, setting the "mark variable" to `GPS_UPDATE_RECEIVED`. This is the default value, and the line could be safely be removed from the configuration block, but it is included anyway to be clear. The PeriodicSurface behavior, once it has noted that the vehicle has reached the surface, will wait until a value has been posted to `GPS_UPDATE_RECEIVED` that is different than its previous posting. Usually a timestamp suffices. The configuration of `uTimerScript` to achieve simulated GPS updates is described in more detail in the `uTimerScript` documentation in the Section "A Script Used as Proxy for an On-Board GPS Unit".

1.6 Topic #4: The Waypoint Behavior with Survey Patterns

The Waypoint behavior, described generally in Section ??, is used in the Delta mission to conduct survey patterns like that shown in Figure 2. The behavior configuration, in file `delta.bhv` is shown in Listing 4 below. Note the survey pattern, described on line 15, does not list a set of points, but instead describes the pattern in terms of its properties such as the lane width and pattern orientation. This format is supported generally for building SegList objects from string specifications, discussed in the Geometry documentation.

Listing 1.4: The Waypoint behavior in the Delta mission, from the file delta.bhv.

```

1 Behavior = BHV_Waypoint
2 {
```

```

3  name      = waypt_survey
4  pwt       = 100
5  condition = MODE==SURVEYING
6  perpetual = true
7  updates   = SURVEY_UPDATES
8  endflag   = SURVEY = false
9
10         lead = 8
11     lead_damper = 1
12         speed = 2.0    // meters per second
13         radius = 8.0
14         points = format=lawnmower, label=dudley_survey, x=80, y=-80,
                    width=70, height=30, lane_width=8, rows=north-south,
                    degs=30
15 }

```

The waypoint behavior is configured to execute the survey pattern once, since the `repeat` parameter is not set and defaults to zero. Upon completion, it posts an end flag, configured on line 8, `SURVEY=false` to the `MOOSDB`. This immediately moves the vehicle out of the `SURVEYING` and into either the `LOITERING` or `RETURNING` mode depending on what it was doing when it was commanded to begin the survey. See the hierarchical mode declarations at the top of the `delta.bhv` file. Since the behavior is set with `perpetual=true` on line 6, completion of the survey pattern merely puts the behavior in a virtual stand-by mode until it is given a new set of waypoints and it once again meets its run conditions.

The waypoint behavior is configured to accept dynamic parameter updates on line 7 through the MOOS variable `SURVEY_UPDATES`. In the Delta mission, updates are initiated by a mouse click in the `pMarineViewer` window, with the result being a post to the `MOOSDB` of the `SURVEY_UPDATES` variable. Such a posting would consist of a new value for the `points` parameter, replacing the initial configuration on line 14. This is discussed next.

1.7 Topic #4: Using pMarineViewer with Geo-referenced Mouse Clicks

In the Delta mission, the `pMarineViewer` application is used to post messages to the `MOOSDB` to (a) put the vehicle into the `SURVEYING` mode from either the `LOITERING` or `RETURNING` modes and (b) grab a user specified point in the operation area, from the user's mouse click, to be used as the center point of the commanded survey pattern. This situation is shown in Figure 2. This is achieved by utilizing the Mouse Context feature of `pMarineViewer`, discussed in the `pMarineViewer` documentation, [?], in the section *"Contextual Mouse Poking with Embedded OpArea Information"*. The below two lines are inserted into the `pMarineViewer` configuration block in `delta.moos`:

```

left_context[survey-point] = SURVEY = true
left_context[survey-point] = SURVEY_UPDATES = points = vname=$(VNAME), \
    x=$(XPOS), y=$(YPOS), format=lawnmower, label=delta, width=70, \
    height=30, lane_width=8, rows=north-south, degs=80

```

Both lines declare that a MOOS variable-value pair is to be posted whenever a left mouse click is generated by the user. The first line sets `SURVEY=true`, in an attempt to put the helm into the `SURVEYING` mode. This alone will not suffice to switch modes if the current value of the MOOS

variable `RETURN` is set to `true`. See the mode declarations near the top of the `delta.bhv` file for this mission - a command to return overrides a command to perform a survey. The second line above associates with a left-mouse click a posting to the `SURVEY_UPDATES` variable. Recall from Listing 4 that this is the very variable through which the surveying waypoint behavior is configured to receive dynamic parameter updates. This posting is configured with a few macros, `$(VNAME)`, `$(XPOS)`, `$(YPOS)`, which are filled in with the name and position of the current vehicle in the `pMarineViewer` window. Each user left-mouse click thus re-assigns the vehicle to a new survey pattern, and switches the helm mode, unless it has been commanded to return.

1.8 Suggestions for Further Experimenting with the Delta Mission

1.8.1 Failing to Reason about Depth

In this mission, *depth* is a mandatory helm decision variable, along with *course* and *speed*, as configured in `delta.moos` and shown in Listing 1. If a mission is not configured properly it's possible to bring about a helm mode where no behavior is reasoning about depth. Declaring *depth* to mandatory is equivalent to declaring that a situation where a helm iteration with no decision on *depth* is an error condition worth of an all-stop.

One way to bring this about in this mission, is to reconfigure the `ConstantDepth` behavior configuration in `delta.bhv` to replace line 6 in Listing 2 with the following

```
condition = (MODE==LOITERING)
```

By doing this, there will be no *depth* related behavior when the vehicle is in the `RETURNING` mode. Try re-running the mission. Note when the mission is first launched, the label next to the vehicle in `pMarineViewer` reads "dudley (PARK)(ManualOverride)". This is normal and indicates that the helm state is `PARK`, simply because the operator retains manual control. In addition to seeing this in the `pMarineViewer` window, this can also be confirmed by scoping on a few key helm variables including `IVPHELM_STATE` and `IVPHELM_ALLSTOP`:

```
$ uXMS delta.moos IVPHELM_STATE IVPHELM_ALLSTOP BHV_ERROR --show=source,time,community
```

should produce something similar to:

VarName	(S)ource	(T)ime	(C)ommunity	VarValue (MODE = SCOPE:PAUSED)
-----	-----	-----	-----	----- (61)
IVPHELM_STATE	pHelmIvP	81.86	dudley	"PARK"
IVPHELM_ALLSTOP	pHelmIvP	1.69	dudley	"ManualOverride"
BHV_ERROR	n/a	n/a	n/a	n/a
MODE	n/a	n/a	n/a	n/a

Note that the `IVPHELM_ALLSTOP` value is posted once, until its value changes, and `IVPHELM_STATE` posts its value on every helm iteration regardless of the value. The latter variable also serves the purpose of a helm heartbeat indicator.

After noting the above, launch the mission by hitting the DEPLOY button, and note that the label next to the vehicle has changed to "dudley (LOITERING)". The four MOOS variables from above have also changed to:

VarName	(S)ource	(T)ime	(C)ommunity	VarValue (MODE = SCOPE:PAUSED)
-----	-----	-----	-----	----- (103)
IVPHELM_STATE	pHelmIvP	109.56	dudley	"DRIVE"
IVPHELM_ALLSTOP	pHelmIvP	98.07	dudley	"clear"
BHV_ERROR	n/a	n/a	n/a	n/a
MODE	pHelmIvP	98.07	dudley	"ACTIVE:LOITERING"

After the vehicle has been running a bit, try hitting the RETURN button which changes the helm to the RETURNING mode. In this case, due to our tinkering with the mission above, there is no behavior reasoning about depth, and the following message appears next to the vehicle on the screen: "dudley (Returning) (MissingDecVars:depth)". The four scoped MOOS variables also change to the following:

VarName	(S)ource	(T)ime	(C)ommunity	VarValue (MODE = SCOPE:PAUSED)
-----	-----	-----	-----	----- (134)
IVPHELM_STATE	pHelmIvP	621.34	dudley	"DRIVE"
IVPHELM_ALLSTOP	pHelmIvP	617.09	dudley	"MissingDecVars:depth"
BHV_ERROR	pHelmIvP	621.59	dudley	"MissingDecVars:depth"
MODE	pHelmIvP	617.09	dudley	"ACTIVE:RETURNING"

Note that, despite the error and all-stop event, the helm remains in drive, and may return to loitering or surveying at any time. Try configuring the helm in `delta.moos` to contain the line `park.on.allstop=true` and note the difference in the above experiment. The generated error will bump the helm into the PARK helm status.

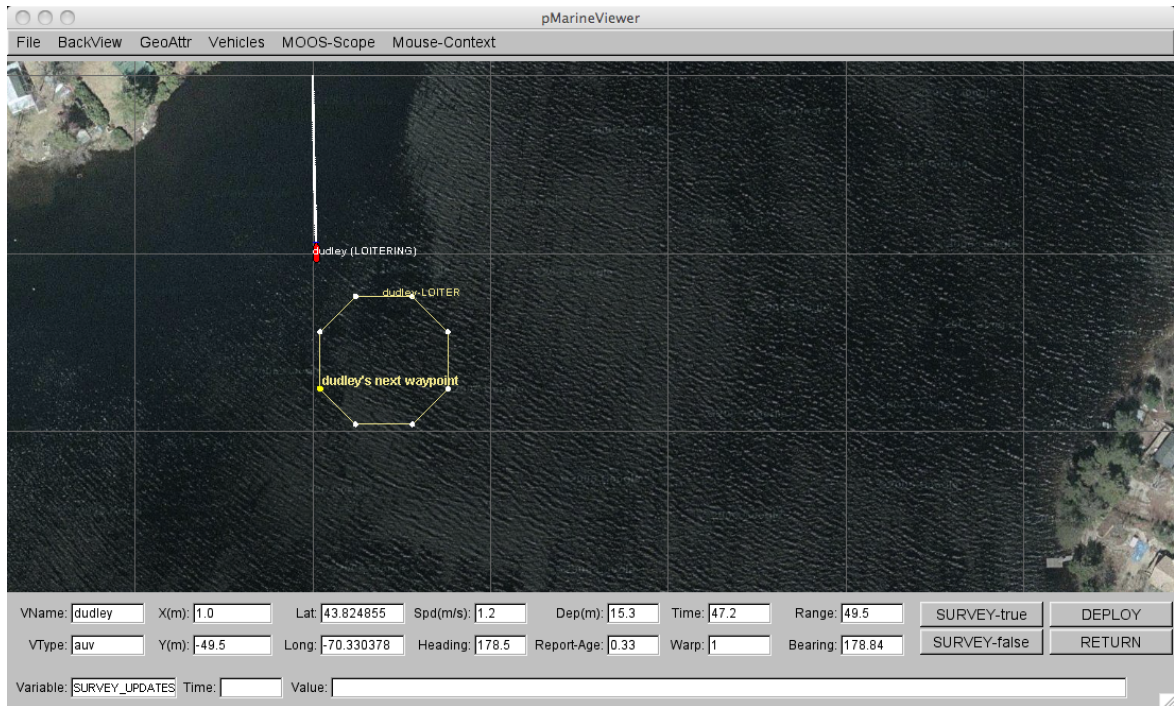


Figure 1: **The Delta Mission (1):** The vehicle "dudley" heads to its loiter region where it will remain indefinitely until it is commanded to return or perform a survey pattern. It will periodically surface for a GPS fix.

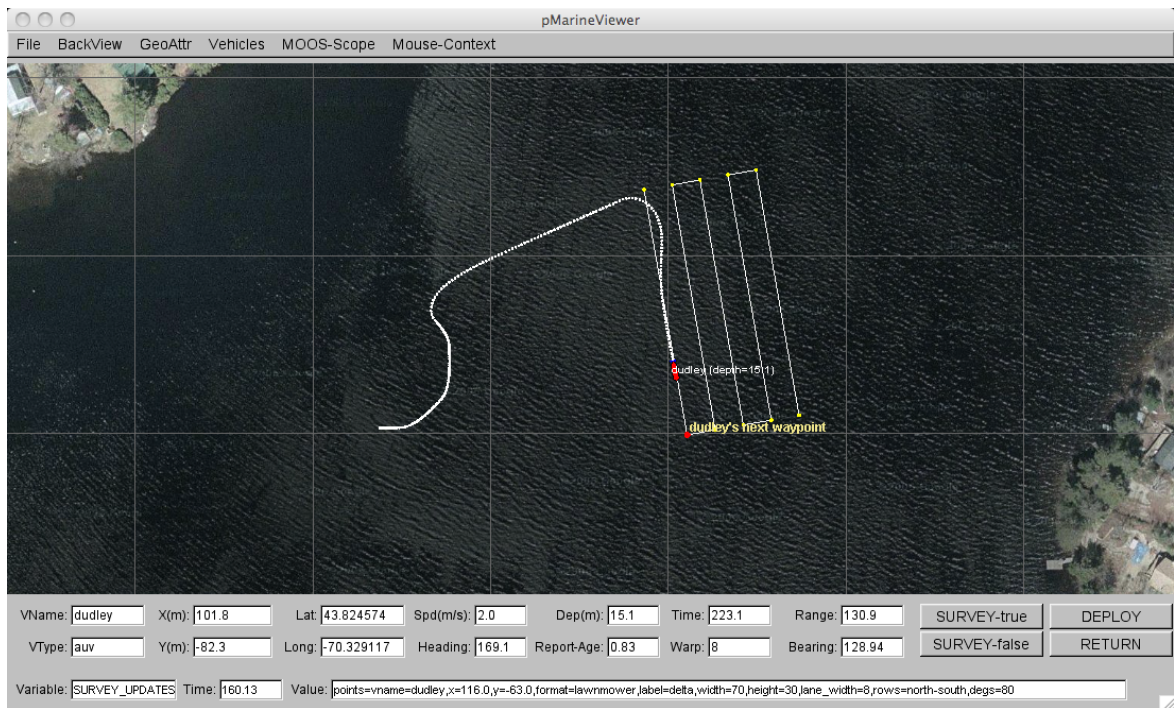


Figure 2: **The Delta Mission (2):** The user has clicked a point in the viewer around which a survey pattern is built. The vehicle exits the loitering mode and begins to execute the survey pattern.

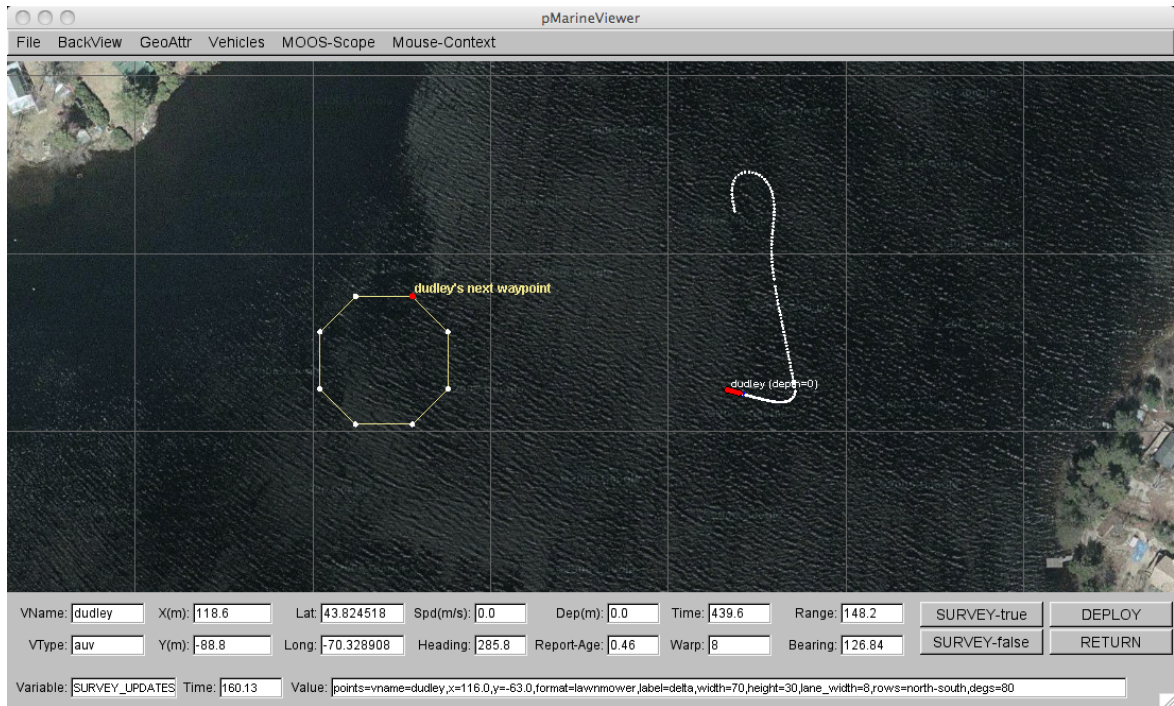


Figure 3: **The Delta Mission (3)**: Once the vehicle has finished the survey pattern, it re-enters the loiter mode returning to its loiter region. It resumes periodic surfacing and immediately surfaces for a GPS fix.

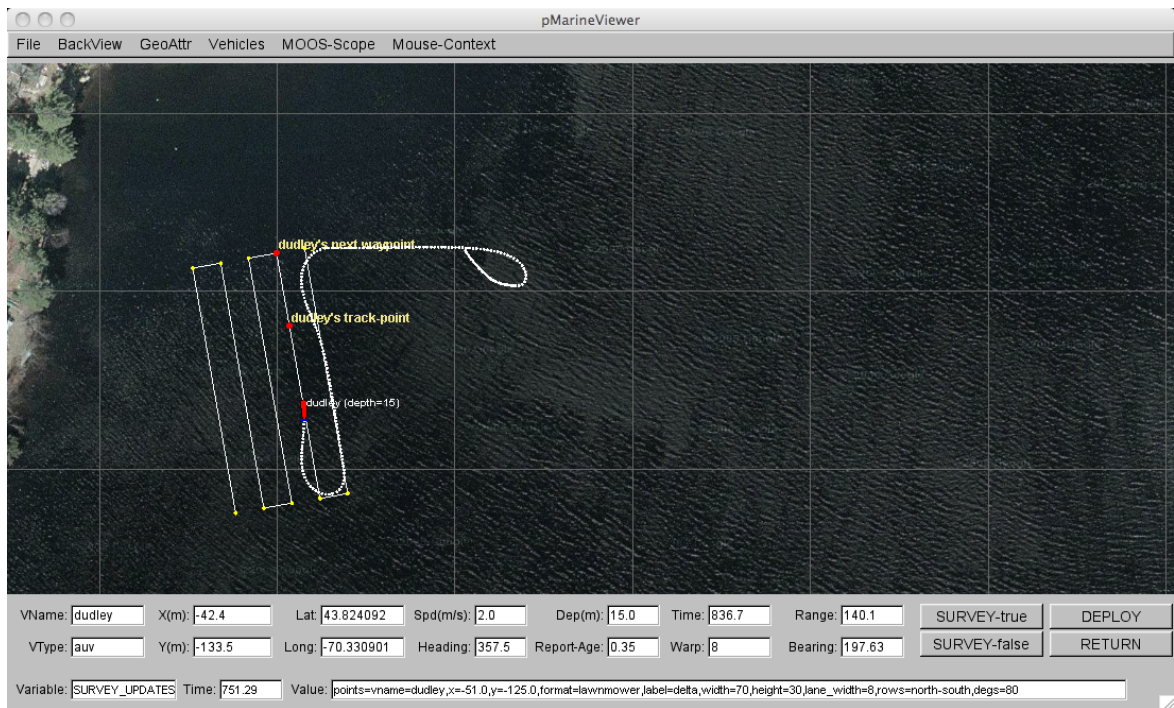


Figure 4: **The Delta Mission (4)**: After resumption of loitering, the user clicks in the viewer a new point around which a new survey pattern is built. The entry point of the survey pattern automatically accommodates the vehicle.

The Echo Mission

Fall 2024

Michael Benjamin, mikerb@mit.edu
Department of Mechanical Engineering
MIT, Cambridge MA 02139
project-pavlab/chapters/chap_xmiss_echo

1	The Echo Mission	1
1.1	Overview of the Echo Mission Components and Topics	1
1.2	Launching the Echo Mission	1
1.3	Topic #1: The BearingLine Behavior	2
1.4	Topic #2 Dynamic Behavior Spawning	3
1.5	Topic #3: Sending Updates to the Original and Spawnd Behaviors	4
1.6	Suggestions for Tinkering	5

Contents

1 The Echo Mission

The primary purpose of the Echo example mission is to illustrate the use of dynamically spawned behaviors. A simple behavior, the BearingLine behavior, is used to illustrate the idea. The BearingLine behavior simply posts a viewable point and viewable line segment representing the bearing from the present position of the vehicle to a fixed point in the operation area. Each new "bearing point" posted to the MOOSDB results in a newly spawned BearingLine behavior in the helm.

1.1 Overview of the Echo Mission Components and Topics

Behaviors:	Waypoint, BearingLine
MOOS Apps:	pHelmIvP, pLogger, uSimMarine, pMarinePID, pNodeReporter, pMarineViewer
Primary Topics:	(1) The BearingLine behavior. (2) Dynamic behavior spawning. (3) Sending updates to the original and spawned behaviors.
Side Topics:	(1) uTimerScript is used to auto-generate events with random components that lead to behavior spawning. (2) uHelmScope may be used to monitor the spawning and death of behaviors in the helm.

1.2 Launching the Echo Mission

The echo mission may be launched from the command line:

```
$ cd moos-ivp/missions/s5_echo/  
$ ./launch.sh --warp=10
```

This should bring up a `pMarineViewer` window like that shown in Figure 1, with a single vehicle, *henry*, initially having the "PARK" helm state. After hitting the DEPLOY button in the lower right corner, the vehicle enters the "SURVEYING" mode and begins to proceed along the waypoints as shown.

1.3 Topic #1: The BearingLine Behavior

In Figure 1, note the line segment rendered from the vehicle in the direction of point (100, -100). This line segment is posted to the `MOOSDB` by the IvP Helm on behalf of the BearingLine behavior. This behavior does not influence the trajectory of the vehicle at all. The line segment acts as an easy visual confirmation that the behavior is instantiated and running properly. In the Echo mission this behavior is configured as in Listing 1. The bearing line originates from the present vehicle position toward the point specified on line 8. The length of the line is 50% of the present distance between the vehicle and the bearing point, as specified on line 7. The bearing point is also configured to be rendered by the configuration on line 9.

Listing 1.1: Configuration of the BearingLine behavior in the Echo example mission.

```

1 Behavior = BHV_BearingLine
2 {
3     name      = bng-line
4     templating = clone
5     updates   = BEARING_POINT
6
7     line_pct = 50
8     bearing_point = 100,-100
9     show_pt = true
10 }
```

The BearingLine behavior produces no (IvP) objective function, so by the definition of the behavior run states (Section ??), it is never in the *active* state. In the Echo mission, the BearingLine behavior is in the *running* state when the vehicle is surveying the five waypoints shown in Figure 1. This can be confirmed by launching a `uHelmScope` window:

```
$ uHelmScope moos-ivp/ivp/missions/s5_echo/echo.moos -x -p}
```

The output should be similar to that shown in Listing 2 below. Note the Waypoint behavior is active, line 12, and the BearingLine behavior is in the running state, line 14.

Listing 1.2: Output of the uHelmScope tool during the execution of the Echo mission.

```

1 ===== uHelmScope Report ===== DRIVE (8)
2 Helm Iteration: 173 (hz=0.25)(5) (hz=0.25)(100) (hz=0.26)(max)
3 IvP functions: 1
4 Mode(s):
5 SolveTime: 0.00 (max=0.01)
6 CreateTime: 0.00 (max=0.01)
7 LoopTime: 0.00 (max=0.01)
8 Halted: false (0 warnings)
```

```

 9  Helm Decision: [speed,0,4,21] [course,0,359,360]
10    course = 189.0
11    speed = 2.0
12  Behaviors Active: ----- (1)
13    waypt_survey (43.0) (pwt=100.00) (pcs=6) (cpu=0.17)
14  Behaviors Running: ----- (1)
15    bng-line (43.0) (upd=0/0)
16  Behaviors Idle: ----- (1)
17    waypt_return
18  Behaviors Completed: ----- (0)
19
20  #  MOOSDB-SCOPE ----- (Hit '#' to en/disable)
21  @  BEHAVIOR-POSTS TO MOOSDB ----- (Hit '@' to en/disable)

```

1.4 Topic #2 Dynamic Behavior Spawning

The Echo mission is configured to allow dynamic behavior spawning for the BearingLine behavior. Lines 4 and 5 in Listing 1 allow the spawning by configuring templating to be enabled on line 4, and specifying the MOOS variable through which spawning requests are received on line 5. Recall that templating can be enabled with either the "clone" or "spawn" options. In this case, "clone" option was chosen to allow the instantiation of one initial instance upon helm start-up, with the parameter configuration shown.

In this example, the `pMarineViewer` is configured to convert left-mouse clicks into posts of the `BEARING_POINT` variable to the `MOOSDB`, triggering the spawning of new BearingLine instances. A mouse click over the point (-5, -58) results in the post:

```
BEARING_POINT = "name=bng-line-5.0--158.0, bearing_point=-5.0,-158.0"
```

The helm receives mail for the `BEARING_POINT` variable since it registers automatically for each variable specified in any behavior configured with the `updates` parameter. The helm examines this string before applying the update, and notes that the behavior name specified is unique (not currently instantiated) and rather than interpreting this as a request to update the existing BearingLine behavior already instantiated, interprets it as a request to spawn a new BearingLine instance if templating is enabled (which it is). The new behavior is spawned, with the behavior name specified. In this case the behavior name is based on the coordinates of the point clicked by the user. Two successive clicks on the same point will result in two posts to `BEARING_POINT` by `pMarineViewer`, but the second post will be effectively ignored by the helm. (It is read by the helm, but since the behavior name is one that is already known to the helm, the update is applied to that existing behavior instance. In this case such an update to the `bearing_point` parameter would be redundant.

After two such user mouse clicks, there will be two new BearingLine behaviors instantiated, and the situation would look similar to that shown in Figure 2. If the `uHelmScope` tool is still connected as above, the output would look similar to that shown in Listing 3. Note the existence of three running BearingLine behavior instances reported in lines 15-17. The instance on line 15 was created upon helm startup, and the instances on lines 16-17 were created upon the user mouse clicks.

Listing 1.3: Output of the uHelmScope tool during the later execution of the Echo mission.


```

1  ===== uHelmScope Report ===== DRIVE (12)
2  Helm Iteration: 279 (hz=0.25)(5) (hz=0.25)(100) (hz=0.26)(max)
3  IvP functions: 1
4  Mode(s): ACTIVE:SURVEYING
5  SolveTime: 0.00 (max=0.01)
6  CreateTime: 0.00 (max=0.01)
7  LoopTime: 0.00 (max=0.01)
8  Halted: false (0 warnings)
9  Helm Decision: [speed,0,4,21] [course,0,359,360]
10 course = 180.0
11 speed = 2.0
12 Behaviors Active: ----- (1)
13 waypt_survey (69.6) (pwt=100.00) (pcs=4) (cpu=0.28)
14 Behaviors Running: ----- (3)
15 bng-line (69.6) (upd=0/0)
16 bng-line-5--58 (66.1) (upd=1/1)
17 bng-line13--124 (56.8) (upd=1/1)
18 Behaviors Idle: ----- (1)
19 waypt_return
20 Behaviors Completed: ----- (0)
21
22 # MOOSDB-SCOPE ----- (Hit '#' to en/disable)
23 @ BEHAVIOR-POSTS TO MOOSDB ----- (Hit '@' to en/disable)

```

1.5 Topic #3: Sending Updates to the Original and Spawned Behaviors

The action associated with the left-mouse click in the Echo mission is configured in the `pMarineViewer` configuration block in `echo.moos` by:

```
left_context[bng_point] = BEARING_POINT = name=bng-line$(X)-$(Y) # bearing_point=$(X),$(Y)
```

Setting the context for the left and right mouse clicks in `pMarineViewer` is achieved by utilizing the Mouse Context feature of `pMarineViewer`, discussed in the `pMarineViewer` documentation, [?], in the section *"Contextual Mouse Poking with Embedded OpArea Information"*. With the above configuration a left-mouse click may result in the following if clicked at the point (-5, -58):

```
BEARING_POINT = name=bng-line-5--58 # bearing_point=-5,-58
```

Updating or changing the prevailing parameters for existing behaviors is possible via the use of the MOOS variable specified in the `updates` parameter for each behavior. The only difference between an update that changes parameters of an existing behavior, and an update that spawns a new behavior is the inclusion of a `name=<behavior-name>` as above. If this component is present in the string posted to the `MOOSDB` and the `<behavior-name>` specifies an existing behavior, then that behavior only will have its parameters updated. If it does not specify an existing behavior, a new behavior will be spawned with the specified name. If the `name=<behavior-name>` component is not included in the posting, then the update will be applied to all behaviors configured to receive updates via that particular MOOS variable.

This case is a bit interesting since all newly spawned behaviors specify the same MOOS variable for receiving updates. Indeed a single poke to the MOOS variable `BEARING_POINT` could result in the simultaneous configuration modification of all instantiated BearingLine behaviors. In the Echo mission, `pMarineViewer` is configured to make the following pokes to the `MOOSDB` via the ACTION pull-down menu:

```
action  = BEARING_POINT = show_pt=true
action+ = BEARING_POINT = show_pt=false
action  = BEARING_POINT = line_pct=0
action  = BEARING_POINT = line_pct=25
action  = BEARING_POINT = line_pct=50
action  = BEARING_POINT = line_pct=75
action+ = BEARING_POINT = line_pct=100
action  = BEARING_POINT = "name=bng-line # line_pct=0"
action  = BEARING_POINT = "name=bng-line # line_pct=50"
action  = BEARING_POINT = "name=bng-line # line_pct=100"
```

The first two actions will turn on or off the rendering of the bearing points posted by each behavior. The next five actions will adjust the rendering length of the posted bearing line by each behavior. The last two actions will adjust the rendering length of the posted bearing line only for the behavior named "bng-line", the behavior spawned at the time of helm start-up.

1.6 Suggestions for Tinkering

- After the mission is launched, use the ACTION pull-down menu in `pMarineViewer` to select `AUTO_SPAWN=true`. This enables a script via `uTimerScript` that automatically generates new bearing points, spawning new behaviors. These behaviors have their durations set randomly by the script to be in range [5, 10] seconds. Note the new bearing lines emerging and moving with the vehicle until the behavior dies. The script will lead to the spawning (and death) of 5000 behaviors over the course of about two hours of simulation at `MOOSTimeWarp=1`.
- With `AUTO_SPAWN=true` as above, let the vehicle return to the launch point (by clicking the RETURN button in the `pMarineViewer` window. After it reaches the return point and perhaps sits for a bit, hit the DEPLOY button once again to return the vehicle into its SURVEYING mode. Notice that initially there are many bearing lines rendered before returning to only a handful of bearing lines after a few seconds of simulation. This is because newly spawned behaviors do not start their duration clock until the first time the behavior enters the running state. All behaviors spawned while returning to the start point are effectively put on hold until the vehicle is re-deployed. Then they all start their duration clocks simultaneously and all die off 5-10 seconds later.

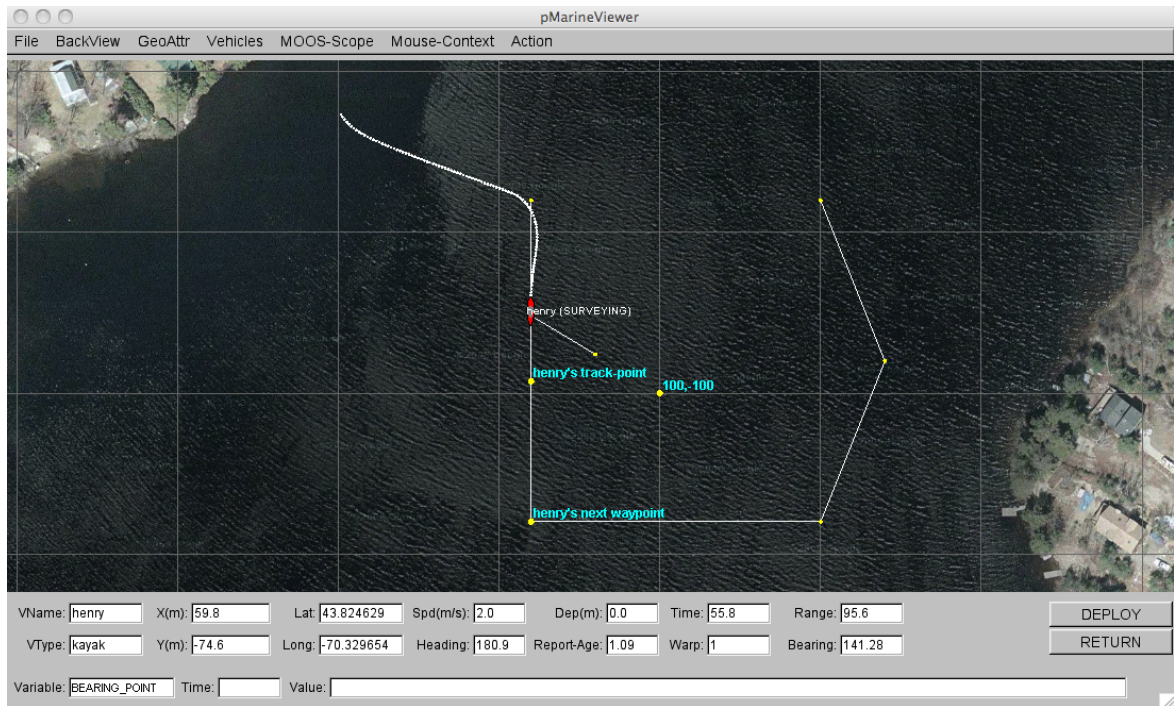


Figure 1: **The Echo Mission (1):** The vehicle "henry" traverses waypoints with the Waypoint behavior. The BearingLine behavior generates a viewable "bearing point" at (100, -100), and a viewable "bearing line".

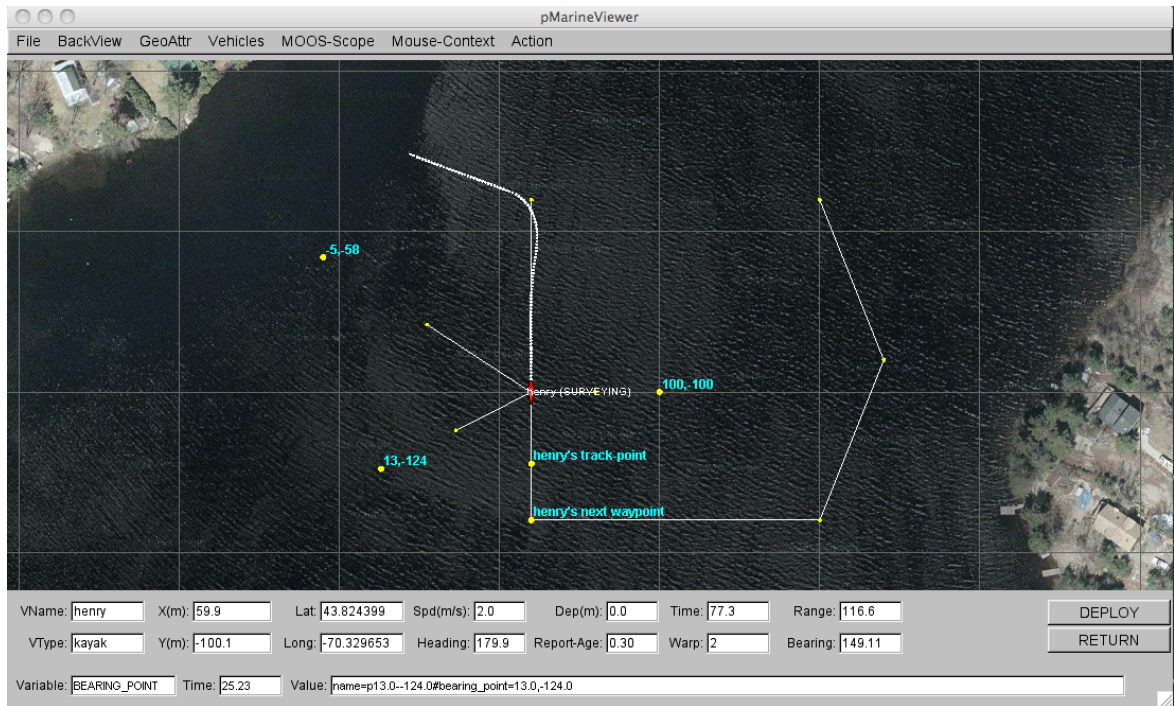


Figure 2: **The Echo Mission (2):** The user has clicked two new bearing points, (-5, -58), and (13, -124), and two new BearingLine behaviors have been spawned, each generating bearing line and bearing point visual outputs.

The Kilo Mission

Fall 2024

Michael Benjamin, mikerb@mit.edu
Department of Mechanical Engineering
MIT, Cambridge MA 02139
project-pavlab/chapters/chap_xmiss_kilo

1 Mission S11: The Kilo Mission	1
1.1 Overview of the Kilo Mission Components and Topics	1
1.2 Launching the Kilo Mission	1
1.3 Topic #1: The Use of a Standby Helm	2
1.4 Topic #2: The TestFailure Behavior	3
1.5 Topic #3: Scoping the Helm State(s) at Runtime	4
1.6 Suggestions for Tinkering	5

Contents

1 Mission S11: The Kilo Mission

The purpose of the Kilo mission is to illustrate the use of the standby helm, and the use of the TestFailure behavior. The standby helm consists of an otherwise normally configured helm with the additional standby parameter invoked. This helm will wait in standby mode until the heartbeat of another (primary) helm ceases to be posted to the MOOSDB for some period of time. In this example mission, a primary and standby helm are configured with the primary helm executing a simple mission similar to the Alpha mission, but also using an instance of the TestFailure behavior. This behavior will be configured to trigger either a crash or hang of the primary helm to demonstrate the manner in which a standby helm will step in to take over with its own mission. The standby mission in this case is simply a behavior to return the vehicle to its launch position.

1.1 Overview of the Kilo Mission Components and Topics

Behaviors:	Waypoint, TestFailure
MOOS Apps:	pHelmIvP, pLogger, uSimMarine, pMarinePID, pNodeReporter, pMarineViewer
Primary Topics:	(1) The use of a standby helm. (2) The TestFailure behavior. (3) Scoping the primary and standby helm states at runtime.
Side Topics:	(1) uHelmScope may be used to monitor the relationship between the shadow helm and primary helm.

1.2 Launching the Kilo Mission

The Kilo mission may be launched from the command line:

```
$ cd moos-ivp/missions/s11_kilo/  
$ ./launch.sh --warp=10          (or more simply, ./launch.sh 10)
```

This should bring up a `pMarineViewer` window like that shown in Figure 4, with a single vehicle, "kilo", initially in the "PARK" mode. After hitting the DEPLOY button in the lower right corner, the vehicle enters the "SURVEYING" mode and begins to proceed along the waypoints as shown.

1.3 Topic #1: The Use of a Standby Helm

The vehicle in this example is configured with both a primary helm and a secondary helm. Configuring a mission with two helm instances is straight forward. The primary helm is configured as done normally, as in lines 3-11 in Listing 1 below. The standby helm is configure identically but with one additional line, as in line 23 in Listing 1 below. This line denotes the helm it to be a *standby* helm, and indicates the time threshold used in determining when to take over from a primary helm.

Listing 1.1: Configuration of the primary and secondary helm in the Kilo mission.

```
1  //----- pHelmIvP config block  
2  
3  ProcessConfig = pHelmIvP  
4  {  
5      AppTick      = 4  
6      CommsTick    = 4  
7  
8      behaviors    = kilo.bhv  
9      domain       = course:0:359:360  
10     domain       = speed:0:4:21  
11 }  
12  
13 //----- pHelmIvP_Standby config block  
14  
15 ProcessConfig = pHelmIvP_Standby  
16 {  
17     AppTick      = 4  
18     CommsTick    = 4  
19  
20     behaviors    = kilo_standby.bhv  
21     domain       = course:0:359:360  
22     domain       = speed:0:4:21  
23     standby      = 2  
24 }
```

Both helms are instances of the `pHelmIvP` application, but one of them is named `pHelmIvP_Standby`. The name of the application is not in any way used to put this helm instance into a standby role. It is chosen simply to be different from the primary helm, since the `MOOSDB` requires all connected applications to have unique names, and to be clear when or if debugging the mission.

ched by `pAntler` with the name `pHelmIvP_Standby`. This is done by giving it the alternative name using the `pAntler ExtraProcessParams` parameter as shown on lines 15 and 18 below.

Listing 1.2: Configuration of Antler.


```

1 //-----
2 // Antler configuration block
3
4 ProcessConfig = ANTLER
5 {
6     MSBetweenLaunches = 200
7
8     Run = MOOSDB           @ NewConsole = false
9     Run = pLogger          @ NewConsole = false
10    Run = uSimMarine        @ NewConsole = false
11    Run = pNodeReporter     @ NewConsole = false
12    Run = pMarinePID        @ NewConsole = false
13    Run = pMarineViewer     @ NewConsole = false
14    Run = pHelmIvP          @ NewConsole = true
15    Run = pHelmIvP          @ NewConsole = true, ExtraProcessParams=HParams
16    Run = uProcessWatch     @ NewConsole = false
17
18    HParams=---alias=pHelmIvP_Standby
19 }

```

If the shadow helm is launched independently, not with `pAntler`, it may be launched with:

```
$ pHelmIvP kilo.moos --alias=pHelmIvP_Standby
```

1.4 Topic #2: The TestFailure Behavior

The Kilo mission uses the `TestFailure` behavior to artificially generate a failure of the primary helm. In the Kilo mission it is configured to produce a "hung" helm with the following configuration:

Listing 1.3: Configuration of the TestFailure Behavior.

```

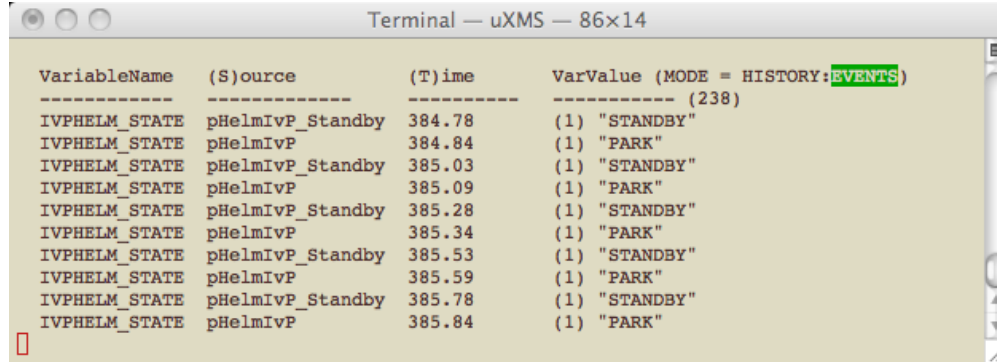
1 //-----
2 Behavior = BHV_TestFailure
3 {
4     name           = test_failure
5     condition      = DEPLOY=true
6     duration       = 120
7     duration_idle_decay = false
8
9     failure_type   = hang,3
10 }

```

The first four configuration parameters are defined for all IvP behaviors. A run condition of `DEPLOY=true` keeps this behavior idle until the mission is launched. Once it is launched, a countdown of 120 seconds begins until the behavior will hang. The `duration_idle_decay=false` setting ensures that the countdown doesn't begin until the behavior is in the running state. In this case the behavior will fail by hanging for three seconds. This will cause the primary helm to also appear to hang for three seconds, noted by a three second gap between heartbeats, defined by posting to the MOOS variable `IVPHELM.STATE`. Since the standby helm is configured to wait no longer than two seconds (see Listing 1), the standby helm will promptly take over control from the primary helm. The sequence of events in taking over control is described in more detail next.

1.5 Topic #3: Scoping the Helm State(s) at Runtime

The relationship between the primary and standby helm may be monitored by scoping on the variable `IVPHELM_STATE` during the course of the mission. Figure 1 below shows the situation shortly after the vehicle is deployed using the `uXMS` scoping tool.



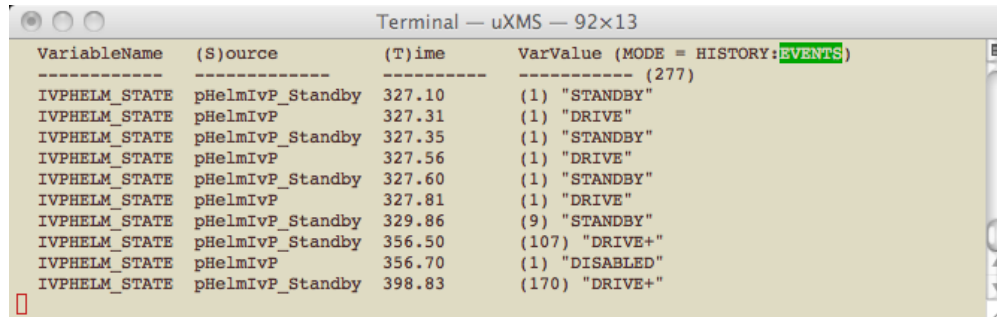
Terminal — uXMS — 86x14

VariableName	(S)ource	(T)ime	VarValue (MODE = HISTORY:EVENTS)
IVPHELM_STATE	pHelmIvP_Standby	384.78	(1) "STANDBY"
IVPHELM_STATE	pHelmIvP	384.84	(1) "PARK"
IVPHELM_STATE	pHelmIvP_Standby	385.03	(1) "STANDBY"
IVPHELM_STATE	pHelmIvP	385.09	(1) "PARK"
IVPHELM_STATE	pHelmIvP_Standby	385.28	(1) "STANDBY"
IVPHELM_STATE	pHelmIvP	385.34	(1) "PARK"
IVPHELM_STATE	pHelmIvP_Standby	385.53	(1) "STANDBY"
IVPHELM_STATE	pHelmIvP	385.59	(1) "PARK"
IVPHELM_STATE	pHelmIvP_Standby	385.78	(1) "STANDBY"
IVPHELM_STATE	pHelmIvP	385.84	(1) "PARK"

Figure 1: **Helm State in the Kilo Mission:** Both the standby and primary helms are operating normally, posting a helm state message about four times per second.

Initially the standby helm is showing a helm state of "STANDBY", and the primary helm is showing a helm state of "DRIVE". Since both helms are operating at the same frequency, they mostly alternate between postings as shown above.

The events shown in Figure 2 show that during the 9 identical postings ending at time 329.86, the standby helm is the only helm emitting a heartbeat. During this period the primary helm has either crashed or hung. Finally the secondary helm takes over and posts a helm state of "DRIVE+" for 107 consecutive heartbeats (iterations). Recall the "+" is to further distinguish that this helm was originally configured as a standby helm.



Terminal — uXMS — 92x13

VariableName	(S)ource	(T)ime	VarValue (MODE = HISTORY:EVENTS)
IVPHELM_STATE	pHelmIvP_Standby	327.10	(1) "STANDBY"
IVPHELM_STATE	pHelmIvP	327.31	(1) "DRIVE"
IVPHELM_STATE	pHelmIvP_Standby	327.35	(1) "STANDBY"
IVPHELM_STATE	pHelmIvP	327.56	(1) "DRIVE"
IVPHELM_STATE	pHelmIvP_Standby	327.60	(1) "STANDBY"
IVPHELM_STATE	pHelmIvP	327.81	(1) "DRIVE"
IVPHELM_STATE	pHelmIvP_Standby	329.86	(9) "STANDBY"
IVPHELM_STATE	pHelmIvP_Standby	356.50	(107) "DRIVE+"
IVPHELM_STATE	pHelmIvP	356.70	(1) "DISABLED"
IVPHELM_STATE	pHelmIvP_Standby	398.83	(170) "DRIVE+"

Figure 2: **Helm State in Kilo Mission after Standby Helm Takes Over Hung Primary Helm:** The standby helm has detected a delay in the primary helm heartbeat and takes over about two seconds later. Note the number in parentheses is the number of identical postings with the timestamp showing the time of the last of the identical postings.

Eventually it turns out that the original primary helm did not crash after all but was just temporarily hung. After emerging from its hung state, upon reading mail in the next loop, it realizes the standby helm has taken over and immediately concedes control and posts the "DISABLED" helm state, and

will never again post. One may wonder why the primary helm, when it finally woke up, did not first mistakenly post "DRIVE" before conceding control - after all it needs to first read its mail to learn that another helm is in control. The primary helm in fact did post "DRIVE" and "DISABLED" on two consecutive iterations. The first posting occurred (time 327.81) prior to querying the behaviors for input, and the second posting (time 356.70) occurred on the next iteration of upon reading mail.

In the case where the standby helm takes over for a crashed helm, the sequence of events, viewed from the perspective of posts to `IVPHELM.STATE`, is similar as shown in Figure 3. When things are going fine the standby helm and primary helm alternately post helm states of `STANDBY` and `DRIVE` respectively as the first several posts below show. At some point the primary helm crashes and the only posts are made by the standby helm. In the example here, there are 10 `IVPHELM.STATE="STANDBY"` posts made while the standby helm is getting closer to triggering a take-over. Finally, upon takeover, it posts a helm state of "DRIVE+" thereafter. The primary helm is never heard from again.

VariableName	(S)ource	(T)ime	VarValue (MODE = HISTORY:EVENTS)
IVPHELM.STATE	pHelmIvP_Standby	162.19	(1) "STANDBY"
IVPHELM.STATE	pHelmIvP	162.30	(1) "DRIVE"
IVPHELM.STATE	pHelmIvP_Standby	162.44	(1) "STANDBY"
IVPHELM.STATE	pHelmIvP	162.55	(1) "DRIVE"
IVPHELM.STATE	pHelmIvP_Standby	162.68	(1) "STANDBY"
IVPHELM.STATE	pHelmIvP	162.80	(1) "DRIVE"
IVPHELM.STATE	pHelmIvP_Standby	162.94	(1) "STANDBY"
IVPHELM.STATE	pHelmIvP	163.05	(1) "DRIVE"
IVPHELM.STATE	pHelmIvP_Standby	165.45	(10) "STANDBY"
IVPHELM.STATE	pHelmIvP_Standby	192.34	(108) "DRIVE+"

Figure 3: **Helm State in Kilo Mission after Standby Helm Takes Over Crashed Primary Helm:** The standby helm has detected a delay in the primary helm heartbeat and takes over about two seconds later. Note the number in parentheses is the number of identical postings with the timestamp showing the time of the last of the identical postings.

1.6 Suggestions for Tinkering

- *Analyze a helm takeover from the log files.* Run the Kilo mission again and, as opposed to monitoring things with a live scope as in Figures 1 - 3, let the mission play out. Change directories to the folder where the `.alog` file was created. Take a look at the `IVPHELM.STATE` postings using the `aloggrep` tool. Confirm that the sequence of events described in Figures 1 - 3 are consistent with entries of the log file. Hint: This can be achieved without mission configuration or code modifications.
- *Crashing with helm with a mouse click.* Rather than waiting for the `TestFailure` behavior to run down its clock to failure, configure it to fail immediately upon entering the run state (`duration=0`). But also set an additional run-condition, e.g., `FAIL=true` and configure the `pMarineViewer` to make this posting upon a mouse-click. Test the modified Kilo mission by launching it, and clicking the mouse when ready to see the failure. This can be achieved solely through mission configuration modification, without code modification.
- *Configure a standby helm that finishes a task.* Create a primary and shadow helm where the primary helm is surveying a set of waypoints. The shadow helm should be able to survey the

remaining points when/if the primary helm is taken over. Hint: this will likely require code generation or modification. Extra bonus if achievable solely through mission modification.

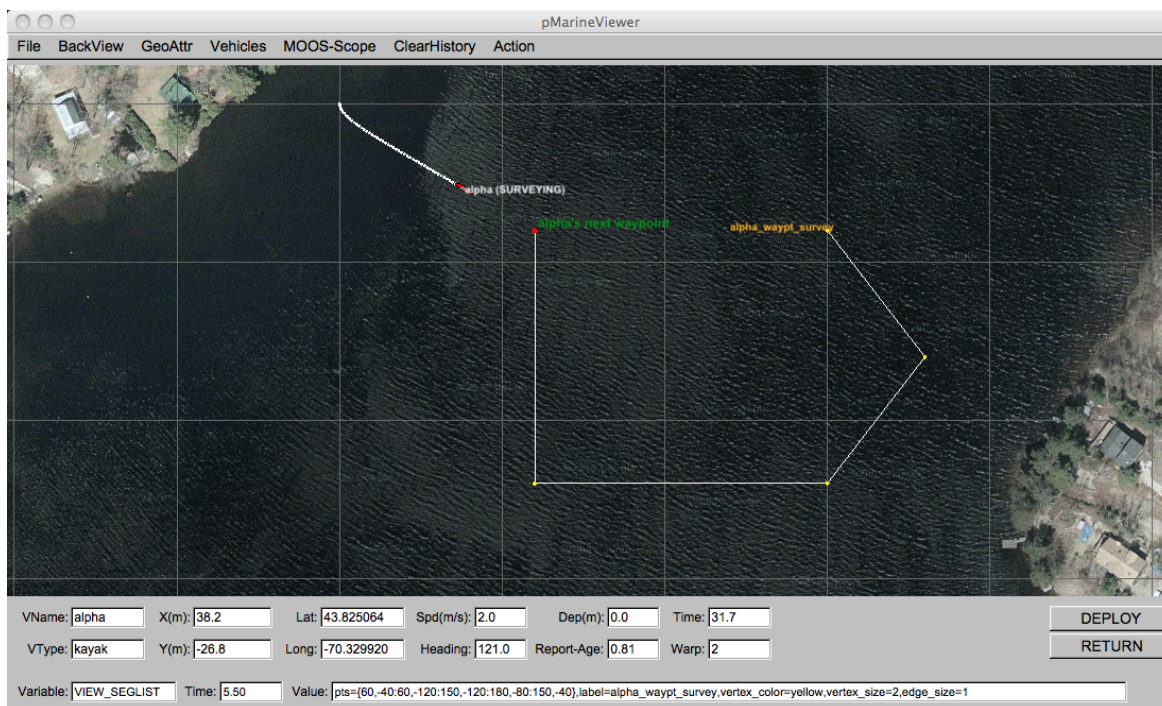


Figure 4: **The Kilo Mission (1):** The vehicle "kilo" traverses the waypoints using a primary helm, launched alongside a standby helm. The TestFailure behavior is a ticking bomb that will hang the primary helm momentarily.

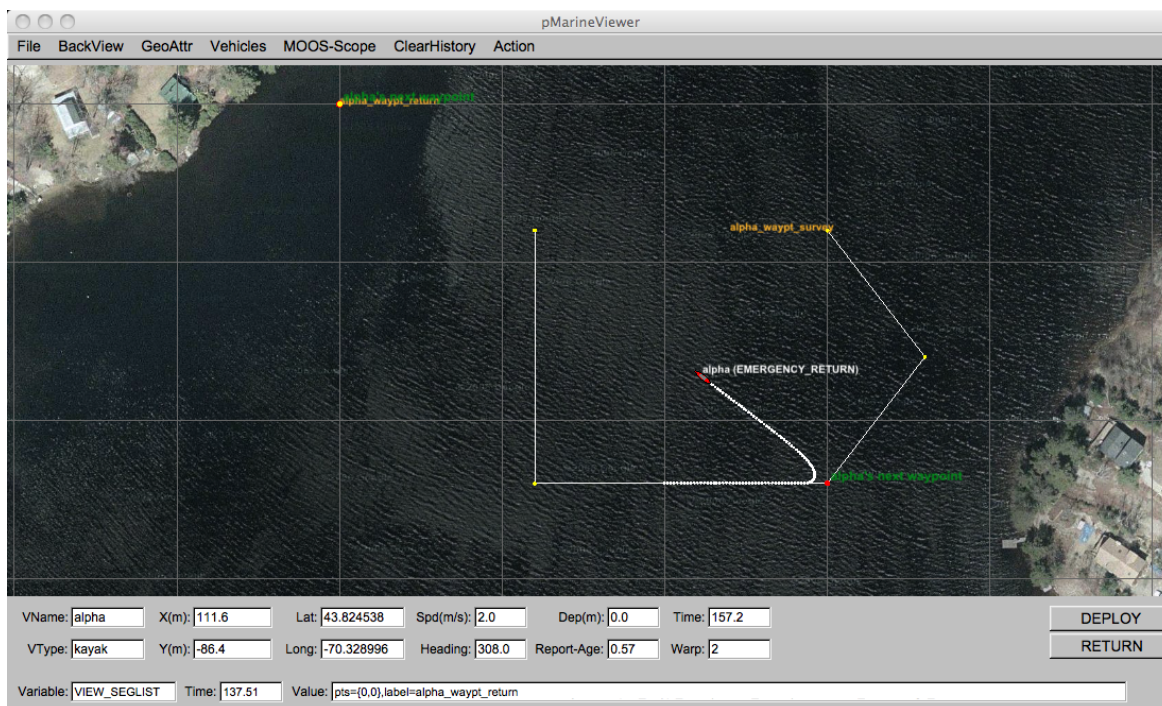


Figure 5: **The Kilo Mission (2):** The TestFailure behavior has hung the primary helm. The standby helm has detected the absence of a heartbeat (the `IVPHELM_STATE` variable), and take over with a simple return mission.

The Berta Mission

Fall 2024

Michael Benjamin, mikerb@mit.edu
Department of Mechanical Engineering
MIT, Cambridge MA 02139
[project-pavlab/chapters/chap_xmiss_berta](https://github.com/project-pavlab/chapters/chap_xmiss_berta)

1	The Berta Mission	1
1.1	Overview of the Berta Mission Components and Topics	1
1.2	Launching the Berta Mission	2
1.3	Topic #1: The AvoidCollision Behavior and Dynamic Behavior Spawning	3
1.4	Topic #2: Contact Management with pContactMgrV20	5
1.5	Suggestions for Tinkering in the Berta Mission	6

Contents

1 The Berta Mission

The Berta mission involves two vehicles repeatedly performing collision avoidance with one another. The primary capabilities highlighted are the helm's collision avoidance behavior, the basic contact manager, `pContactMgrV20`, and the dynamic spawning and removal of behaviors related to contacts from the helm.

1.1 Overview of the Berta Mission Components and Topics

Here is a bit of what the Berta mission should look like:



Figure 1: The Berta mission.

video:(0:43): <https://vimeo.com/84492574>

Behaviors:	Loiter, AvoidCollision
Vehicles:	<i>henry</i> and <i>gilda</i>
MOOS Apps:	<code>pHelmIvP</code> , <code>pLogger</code> , <code>uSimMarine</code> , <code>pContactMgrV20</code> , <code>pMarinePID</code> , <code>pNodeReporter</code> , <code>pMarineViewer</code> , <code>uTimerScript</code> , <code>pShare</code>
Primary Topics:	(1) The IvP Helm AvoidCollision behavior / Dynamic behavior spawning. (2) Contact management with the <code>pContactMgrV20</code> application.
Side Topics:	(1) <code>uTimerScript</code> is used in a command and control role for random permuting of vehicle loiter assignments. (2) <code>pShare</code> is used for connecting one shoreside command and control community to two simulated vehicle communities. (3) The Loiter behavior is configured to receive dynamic updates on its loiter position.

1.2 Launching the Berta Mission

The *Berta* mission may be launched from the command line:

```
$ cd moos-ivp/missions/m2_berta/
$ ./launch.sh --warp=10
```

This should bring up a `pMarineViewer` window like that shown in Figure 2, with two vehicles, *henry* and *gilda*, each initially in the "PARK" state. After hitting the DEPLOY button in the lower right corner, the vehicle enters the "LOITERING" mode and begins to proceed along the loiter pattern as shown. The example mission is configured to periodically alter the position of each vehicle's loiter

pattern, alternating between a random point in Region-A and Region-B shown in Figure 2. The user may click the PERMUTE-NOW button to immediately cause a new permutation and command sent to the vehicle, which otherwise happens automatically every three minutes.

1.3 Topic #1: The AvoidCollision Behavior and Dynamic Behavior Spawning

The AvoidCollision behavior runs on both vehicles and is configured identically on each vehicle in this mission. Its configuration is shown in Listing 1. When the helm first launches on each vehicle this behavior is not present. It is configured to be a dynamically spawned behavior - one for each known contact within a certain range of the vehicle. In this regard it depends on alerts from a contact manager running separately on each vehicle that posts the MOOS variable `CONTACT_INFO`, with the name of the new contact for which to spawn a new behavior.

The `CONTACT_INFO` variable is the variable for which the behavior is configured to receive dynamic updates, on line 6. The behavior is configured to allow dynamic instantiations on line 8 with `templating = spawn`. The choice of `spawn` (vs. `clone`) means that a behavior of this configuration is not initially present unless cued via the `updates` interface. See the discussion on dynamic behavior spawning in Section ??.

Listing 1.1: Configuration of the AvoidCollision behavior in the Berta example mission.

```

1 Behavior = BHV_AvoidCollision
2 {
3     name      = avd_collision
4     pwt       = 200
5     condition = AVOID=true
6     updates   = CONTACT_INFO
7     endflag   = CONTACT_RESOLVED = ${CONTACT}
8     templating = spawn
9
10         contact = to-be-set
11     on_no_contact_ok = true
12     extrapolate = true
13     decay = 30,60
14
15     pwt_outer_dist = 50
16     pwt_inner_dist = 20
17     pwt_grade = linear
18     completed_dist = 75
19     min_util_cpa_dist = 8
20     max_util_cpa_dist = 25
21     bearing_line_config = white:0, green:0.65, yellow:0.8, red:1.0
22 }
```

The parameters in lines 15-17 determine the level of activity of the behavior based on the range to the contact. The parameters in lines 15 and 16 refer to the priority weight of the behavior. Beyond 50 meters in range, the weight will be 0% of its static weight of 200 assigned on line 4. (A weight of zero means that no objective function will be produced and the behavior is in the running state, but not the active state.) At 20 meters range, it will be at 100% of its static priority weight. The grade in priority between outer and inner distances is linear, due to the setting on line 17. The `min_util_cpa_dist`, and `max_util_cpa_dist` on lines 19 and 20 refer to the utilities assigned to

candidate maneuvers in the objective function output by the behavior. Any candidate maneuver whose CPA is less than 8 meters will be given the lowest utility rating, equivalent to an actual collision. Likewise, any candidate maneuver whose CPA is greater than 25 will be given the highest utility rating, equivalent to missing the contact by infinite distance.

The `completed_dist`, on line 18, refers to the range to the contact beyond which the behavior will declare itself to be complete. Once completed, the behavior will post its end flags (none in this case), and will remove itself from the helm. In this regard, some coordination with the contact manager is needed. At the very least, the contact manager must be configured to post alerts to the mutually configured MOOS variable, `CONTACT_INFO`, on line 6 in Listing 1 and on line 16 in Listing 2. Note also that the `completed_dist` is set to 75 meters, which is higher than the 55 meters set for the `alert_range` in the contact manager. This means that, after the avoid collision behavior dies, the contact must come back within 55 meters before the contact manager generates another alert, triggering the helm to once again spawn a behavior for the contact.

Once a contact has gone out of range to the point where the AvoidCollision behavior dies, it typically never returns, in practice. There is no guarantee of this however, since the future maneuvers of either the contact or ownship may once again put them on a collision course. The Berta mission tests this scenario repeatedly. The two vehicles are configured to loiter in a loiter pattern that keeps them out range from each other in terms of triggering an AvoidCollision behavior in each of their helms (as in Figure 2). In this mission, a `uTimerScript` script is running in the shoreside community that periodically picks a new loiter location within one of the two regions shown in Figure 2. It picks a new location once every three minutes, alternating between the two regions for each vehicle each time. This ensures that there will be a reasonably unpredictable collision avoidance situation each time the vehicles transit to their new loiter locations. The permutations happen automatically once every three minutes, but the user may jump to the next permutation by clicking on the `PERMUTE-NOW` button.

In Figure 3, note the line segment rendered between the two vehicles. This line segment is posted to the MOOSDB by the IvP Helm on behalf of the AvoidCollision behavior. A white line is drawn between the two vehicles as soon as the AvoidCollision behavior is spawned. The line turns a non-white color as soon as the behavior begins to generate an objective function, thus actively influencing the helm to prefer maneuvers with a lower expected CPA. The line is initially green to indicate a lower behavior priority. As the priority grows, the line turns yellow and finally red when it is at the higher priorities. The behavior posting of these line segments is optional and configurable. The configuration on line 21 in Listing 1 is responsible for the postings in this mission.

In this example, the `extrapolate` option is turned on for the AvoidCollision behavior (line 12 in Listing 1). Extrapolation is typically enabled when long durations are experienced between updates on contact position (which is not the case here in simulation). The behavior will automatically use the linear extrapolated position given the contact's last known position, trajectory and time-stamp. The linear extrapolation will decay down to a zero speed (stationary solution) beginning at 30 seconds and coming to a complete stop at 60 seconds, due to the decay specification on line 13 of Listing 1.

1.4 Topic #2: Contact Management with pContactMgrV20

In the Berta mission, the `pContactMgrV20` is running on both simulated vehicles, configured identically with the configuration block shown in Listing 2. Both the contact manager and the helm are receiving `NODE.REPORT` messages containing information about contacts. The helm stores this information in its information buffer for any behavior that requests it during normal operation. The contact manager uses contact information to generate alerts, which potentially cues the helm to spawn new contact-related behaviors.

The contact manager may be configured to generate more than one alert. However, in the Berta mission, a single alert, on line 15 in Listing 2 is sufficient for spawning collision avoidance behaviors. Note the alert specifies a variable name, and variable value. The variable name, (`CONTACT_INFO`), is set to be the variable used for updates in the AvoidCollision behavior, on line 6 in Listing 1. The variable value is configured to be #-separated list of parameter-value pairs suitable as input to any behavior receiving updates. (See Section ?? on behavior updates.) In this case, the alert is configured to "update" the AvoidCollision behavior parameters `name`, and `contact`. Both parameter values are comprised, in part, of the name of the contact, `[$VNAME]`. In the case of the behavior name, this is to ensure that the behavior name is unique, a condition for spawning a new behavior. It also ensures that multiple AvoidCollision behaviors will not be spawned for the same contact. In the case of the contact name, the behavior simply must know the name of the contact for which it avoiding a collision.

In the unlikely event that two distinct contacts are present in the field, and reporting the same vehicle name, it would be up to the contact manager to discern that these are indeed two contacts and not one. Currently this is beyond the algorithms implemented in `pContactMgrV20`.

Listing 1.2: Configuration of the `pContactMgrV20` application in the Berta example mission.

```
1 // pContactMgrV20 MOOS Config Block
2
3 ProcessConfig = pContactMgrV20
4 {
5     AppTick    = 2
6     CommsTick  = 2
7
8     alert_range      = 55
9     alert_cpa_range  = 70
10    alert_cpa_time   = 30
11    contact_max_age  = 3600
12    display_raddii   = false
13    verbose          = true
14
15    alert = var=CONTACT_INFO, val="name=avd_[$VNAME] # contact=[$VNAME]"
16 }
```

In the Berta example mission, the contact manager is configured to generate alerts when a known contact comes within 55 meters of the vehicle (line 8 of Listing 2). The contact manager also is configured to generate alerts when a contact comes within 70 meters (line 9) *and* its calculated CPA over next 30 seconds (line 10) falls within the alert range (again line 8). See the `pContactMgrV20` documentation for more on the relationship between these two ranges.

The contact manager is capable of optionally posting to the MOOSDB two circles rendering the two ranges, the `alert_range` and `alert_cpa_range`. The rendering is turned off and on with the setting on line 12. The circles are posted under the variable `VIEW_POLYGON` in the local MOOSDBs on each vehicle, and each vehicle is configured to bridge this variable (via `pShare`) to the shoreside community running the `pMarineViewer`.

1.5 Suggestions for Tinkering in the Berta Mission

- Alter the alert ranges in the contact manager, and enable the displaying of range radii (line 8 in Listing 2) and notice when the AvoidCollision behavior is spawned, by the presence of the bearing lines between the two contacts.
- Alter the `pwt_*_dist` parameters in the AvoidCollision behavior (lines 15-16 in Listing 1) and check how this affects the ranges between the vehicles as they avoid collisions.



Figure 2: **The Berta Mission (1):** The two vehicles, *henry* and *gilda*, initially transit to their respective loiter stations and await further random location changes between the shown boxes.

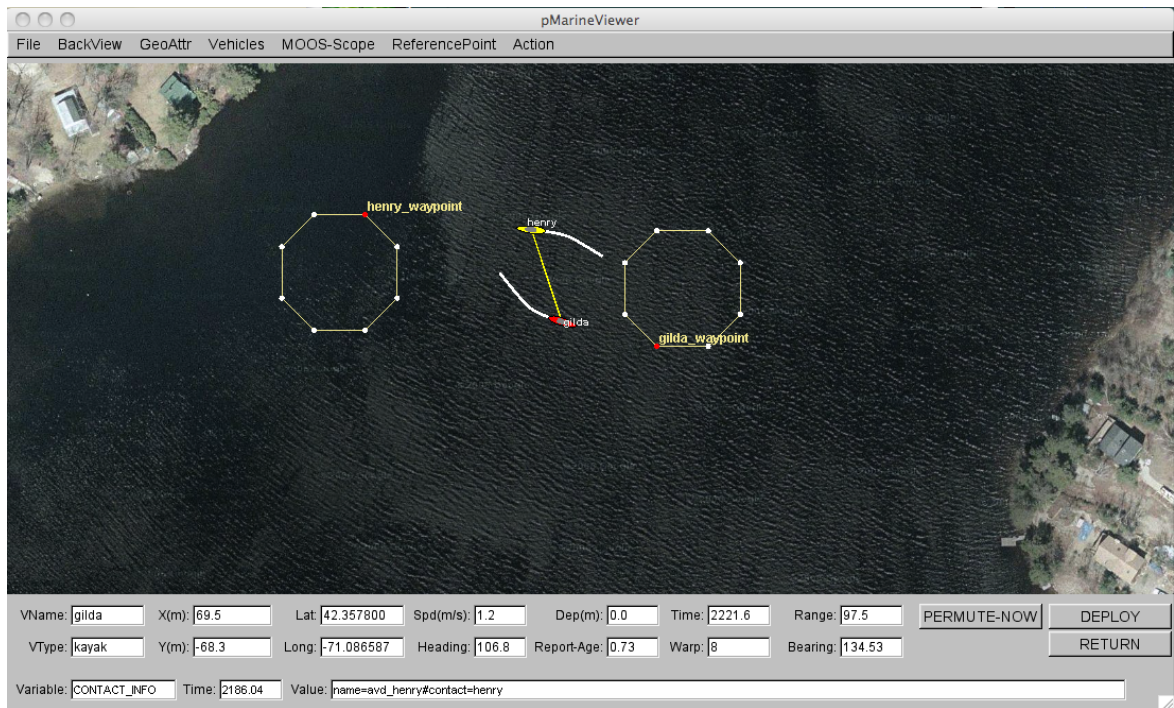


Figure 3: **The Berta Mission (2):** The vehicles have changed their loiter stations and this puts them at risk for collision. The green bearing line between the vehicle indicates the presence of the AvoidCollision behavior.